

WEB UYGULAMA GELİŞTİRME YOL HARİTASI

<https://drive.google.com/file/d/16FSG9unUo1HfYTgH0VRmW9FYby382XPk/view?usp=sharing>

- HTML - CSS

- Kaynaklar

- Eğitim Linkleri (Doküman)

- <https://www.freecodecamp.org/learn/2022/responsive-web-design/> (bu kaynaktaki adımlar sırasıyla takip edilmeli)

- <https://www.w3schools.com/html/default.asp>

- <https://www.w3schools.com/css/default.asp>

- Flexbox = <https://flexboxfroggy.com/>

- Eğitim Video

- (ücretli)

- [:https://www.udemy.com/course/komple-web-developer-kursu/](https://www.udemy.com/course/komple-web-developer-kursu/)

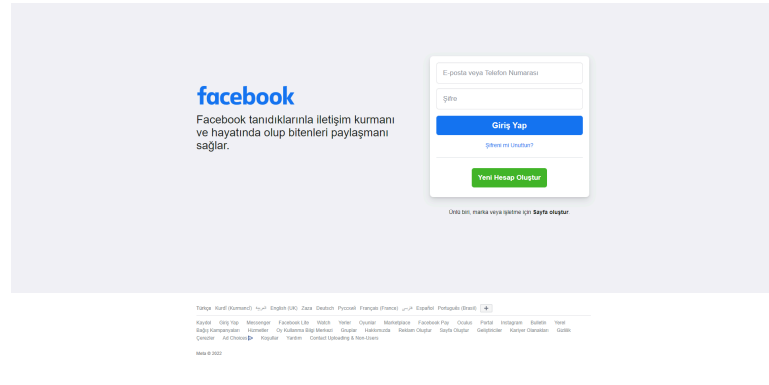
- (ücretsiz):

- [▶ HTML Dersleri - Ders 1 - HTML'ye Giriş ve HT...](#)

- Projeler

- ☐ Facebook giriş ekranı : <https://tr-tr.facebook.com/>

Sayfa responsive tasarıma sahip olmalı,sayfada pixel değerleri kullanılmayacak, % ile değer verilecek.Facebook logosu facebook sayfasından alınması tavsiye edilir.



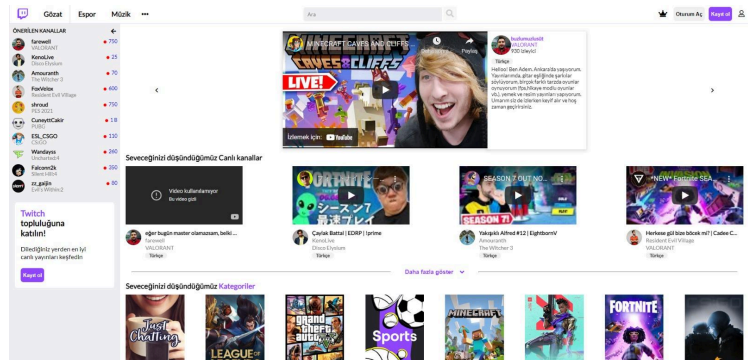
□ **Twitter Giriş ekranı:** <https://twitter.com/>

Sayfa tasarımı responsive tasarım yapısına uygun olmalıdır. Sayfadaki logolar twitter'ın kendi sayfasından alınması tavsiye edilir. Css'de pixel değerleri kullanılmaması gerekir, değerler % ile verilmelidir.



□ **Twitch Anasayfası :** <https://www.twitch.tv/>

Sayfa tasarımı flex yapısına uygun olmalıdır. Css kodlarında classname adları ve id isimleri düzgün adlandırılmalıdır. İsimler kontrol edilecektir. Sayfanın pixeli küçüldükçe sayfadaki yayın kutu sayısı azalmalıdır. (Linkteki sayfanın boyutuyla oynarsanız demek istenileni kolay bir şekilde anlayacaksınız.)



- JAVASCRIPT

- Kaynaklar;

- Döküman

- <https://www.w3schools.com/js/default.asp>

- Video

- (Ücretli) :

- :<https://www.udemy.com/course/komple-web-developer-kursu/>

- (ücretsiz):

-  1 Videoda Javascript Dersleri ile Javascript Pro...

- Fetch API :

-  JavaScript | Fetch API ile Çalışmak #44

Roadmap: <https://x.com/aejazzkhan/status/1855926350848340286>

- Javascript Frameworks

2 seçeneğiniz bulunmaktadır.İkisininide araştırınız ve seçiminizi yapınız.Aynı anda ikisininide öğrenmeye çalışmayın.

- REACTJS

- ANGULAR

- Projeler;

- **Todo List:**

- Üst tarafta bulunan input kutusuna girilen değerin ekleme butonuna basınca aşağı eklenmesi, aşağıda kullanıcı isterse tüm görevleri isterse tek bir görevi silebilmesi gerekmektedir.

Yapılacaklar Listesi

Yeni Görev

Yeni görev girin

+

Görev Listesi

task

Temizle

x

☐ Weather Forecast App:

Projede hedeflenen, kullanıcının aratılan şehrin ismine göre 5 günlük hava durumu bilgisini ekrana getirmesi üzerinedir. Bonus olarak kullanıcının arama yaptığı şehrin bilgisini doğru girmediği zaman uyarı mesajı getirmesi, ve ilgili şehri yansıtacak resmi arka plan olarak göstermesi olabilir. Tasarımın responsive olması amaçlanmaktadır. API için yardımcı kaynak olarak yukarıda bahsedilen youtube linki ve aşağıdaki api linki incelenebilir: <https://api.openweathermap.org/>



☐ Table Generator:

Üst kısımda bulunan inputlara girilen veriler create table butonuna basınca yeni bir tablo oluşturup içine verilerin eklenmesi gerekir. Inputlara girilen veri istenilen tabloya eklenmesi gerekir. İstenilen tablodan istenilen satır silinmesi gerekir. İstenilen tablodan tek bir buton ile tüm satırlar silinmelidir. İstenilen table silinmelidir. Tüm tablolar

tek bir butonla silinmelidir.

□ Personal Information Table;

Projede hedeflenen, kullanıcının personel bilgileri içeren bir ön yüz tasarımı yapması, buna bağlı olarak resimde yer alan id, ad-soyad,tc ve telefon bilgilerini eklemesi, listelemesi, arama gerçekleştirmesi, düzenlemesi ve silmesi üzerinedir. Proje responsive tasarıma sahip olmalı, buna ek olarak girilen alanlar üzerinde validation (imla doğrulaması) yapılması gerekmektedir(Örn. email formatı, tc formatı vs).

- ReactJS

- Kaynaklar;

- Döküman

- <https://tr.reactjs.org/docs/getting-started.html>

- Video

- [ReactJS Eğitim Serisi 1 - Giriş ve ReactJS Ned...](#)

-  Yeni Başlayanlar İçin REACT : 00 : React Nedi...

- AngularJS

- Kaynaklar;

- Döküman

- <https://angular.io/docs>
 - <https://material.angular.io/>
 - <https://medium.com/@batuhanbskr/1-angulara-giri%C5%9F-t%C3%BCrk%C3%A7e-92c70c2cb692>

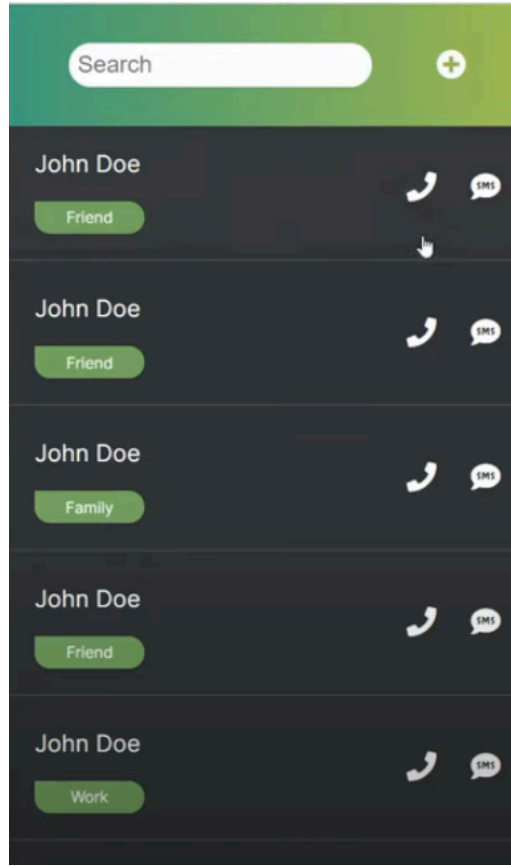
- Video

- (ücretli)
<https://www.udemy.com/course/komple-web-developer-kursu/>
 - (ücretsiz)
 Angular Tutorial for Beginners: Learn Angular &...

- Projeler(Framework Projeleri);

- **Phone-Contact:**

Bu projede hedeflenen form kullanarak CRUD işlemleri(yeni kullanıcı ekleme, düzenleme, silme), Componentler arası veri transferi,search box kullanımına dikkat edilmesi gerekmektedir.Rehber tasarımını web sayfasının ortasına yapmalısınız.



■ Book Store:

Bu projede hedeflenen butonla sayfa tema rengi deęiřtirme ,database ya da array'de tutulan bilgilerin resim olarak ekranda kullanılması aynı zamanda tabloda gösterilmesidir.Resimlerin internetten canlı gelmesi gerekmektedir.(bilgisayardaki dosya uzantısı ile almayınız). Resim ve tablo farklı componentlerde olması gerekmektedir. Her resmin ve tabloda yalnızca elemanların üzerine gelince hover efekti olması css'te istenen özelliktir.

BOOKS STORE

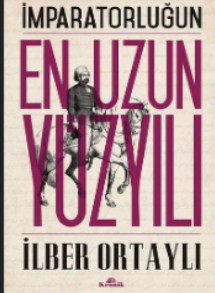
DATE MOD ON



Kırmızı Pazartesi
Gabriel Garcia Marquez



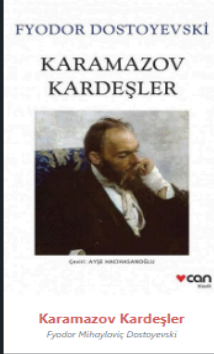
Şeker Portakalı
Jose Muro de Vasconcelos



En Uzun Yüzyıl
İlber Ortaylı



Dracula
Bram Stoker



Karamazov Kardeşler
Fyodor Mihayloviç Dostoyevski

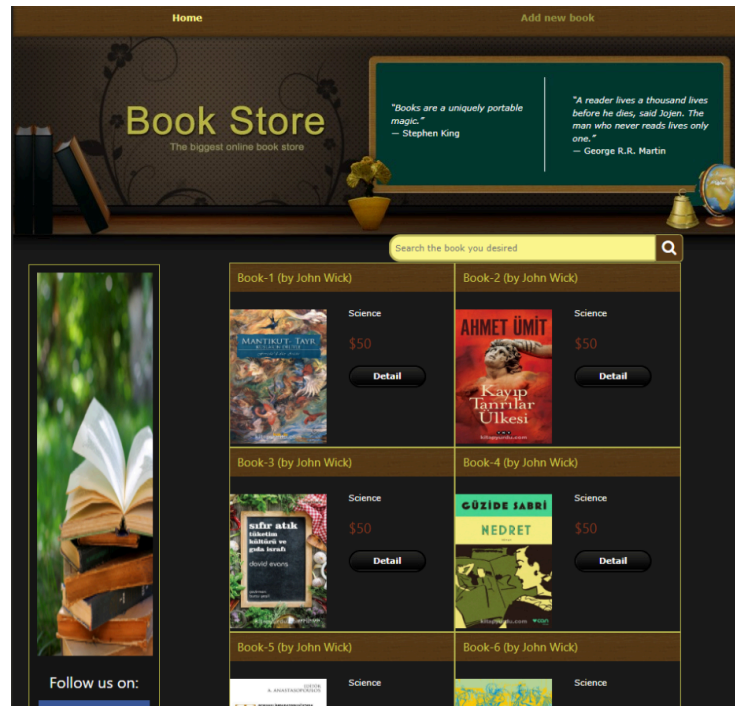


Sultanın Korsanları
Emrah Safa Gürkan

Number	Name	Author
1	Kırmızı Pazartesi	Gabriel Garcia Marquez
2	Şeker Portakalı	Jose Muro de Vasconcelos
3	En Uzun Yüzyıl	İlber Ortaylı
4	Dracula	Bram Stoker
5	Karamazov Kardeşler	Fyodor Mihayloviç Dostoyevski
6	Sultanın Korsanları	Emrah Safa Gürkan

■ Book Store version 2:

Projede hedeflenen, Javascript frameworku kullanılarak kitapçı site arayüzü tasarlanması hakkındadır. Buna ek olarak arayüzde yeni bir kitap ekleme ve listelenen kitaplar için arama yapma ve ilgili kitabın detaylarını gözlemleme üzerinedir.



- NodeJS
 - Kaynaklar;
 - Döküman;
 - <https://node-postgres.com/api/client>
 - <https://nodejs.org/en/docs/>
 - <https://www.robinwieruch.de/node-express-server-rest-api/>
 - Video
 - [Node js Dersleri 1.Ders Giriş](#)
 - Projeler;
 - CRUD Operation
 - REST API

API ve fullstack

<https://www.youtube.com/watch?v=b3SL2S1zYwU>

Başka branşlardan gelen ve bilgisayar mühendislik formasyonu almak isteyenler için;

Bilgi Teknolojileri Sertifika Programı, Onl ne

Ortadoęu Teknik  niversitesi

Program s resince alınan dersler ve bu dersleri veren eęitmenler,

- Bilgisayar Sistemleri ve Python Programlama: Dr.  zg r Kaya
- Java ile Bilgisayar Programcılıęına Giriş: Prof. Dr. Erman Y kselt rk
- Java ile Veri Yapıları ve Algoritmalar: Prof. Dr.  . Hakkı Toroslu, Dr. Cevat Şener
- Unix  le İřletim Sistemleri: Prof. Dr. Faruk Polat
- Veri Tabanı Y netim S stemleri: Prof. Dr. Adnan Yazıcı
- Yazılım M hendislięi: Prof. Dr. Ali H kmet Doęru
- Web Programlama: Prof. Dr. Ahmet Coşar
- Yazılım Geliřtirme Projesi: Prof. Dr. Veysi İřler

Nesneye y nelik programlama

EKSTRALAR

- 1- CSS kısmında Scss çok kullanılıyor ve öğrenilmeli.
- 2- Javascript öğrenenler için çoğu iş yerinde JQuery önemli bir framework.
Öğrenilmeli.(JQuery javascriptin daha kısa yazılan hazır metodların yaptığı bazı işlemlerdir.)
- 3- React.js öğrenecekseniz Redux mantığını kesinlikle öğrenmelisiniz.
- 4- React.js ile Redux mantığını çözdükten sonra Next.js öğrenmek size çok büyük avantaj sağlar.
- 5- Git ve Github kullanmayı kesinlikle öğrenmelisiniz.
- 6-Typescript kesinlikle öğrenmelisiniz.(React.ts şu anda çok fazla kullanılıyor).
- 7-Reusable kodlamayı bilmek size büyük bir avantaj sağlar. İş ortamında yazılan kod ile kendi yazdıklarımız çok farklı. Bu şekilde ilerlemek takım çalışmasına alışmanıza çok katkı sağlayacaktır.
- 8-Module Css öğrenmeli ve Css kodlamalarınızı bu şekilde yazmalısınız.

YAPAY ZEKA - DERİN ÖĞRENME

CS50's Introduction to Artificial Intelligence with Python

<https://cs50.harvard.edu/ai/2023/>

AI Topics

https://docs.google.com/spreadsheets/d/1YocC2ls3iHXHAle36JPUE-hgDfssvp_hlyBXZGACV4G8/edit#gid=1705889116

Deep Learning Derin Öğrenme - Tensorflow

<https://www.tensorflow.org/tutorials>

Nvidia Jetson Nano, Tutorials, Beginners Guide

<https://bilkav.com/makine-ogrenmesi-egitimi/>

https://www.youtube.com/@Sadievrenseker_BK/videos

VERİ BİLİMİ - DATA SCIENCE

?????

SOFTWARE DEVELOPER KEYWORDS

Docker, github, nginx, RESTful, web services

Veritabanları : postgresql, mysql

Cloud : google cloud, aws, microsoft azure

Mobil : react native, flutter

IDE : Visual Studio Code

Bilgisayar / IT Information Technologies

- Programlama, C/C++, C#, Java, Backend/Frontend, PHP, Javascript, Angular, React, Android Studio, NodeJS, ASP, .NET
- Ağ - Network, Kablolama, Switch, Veri Merkezi, Sunucular, Kabinet, Firewall, Siber Güvenlik
- Sistem, Windows, Linux, Sunucular, Web sunucu, Mail sunucu, CI/CD DevOps, Server Admin, Sanallaştırma, Cloud
- Veritabanı, Oracle, SQL Server, PostgreSQL, MySQL, NoSql, Hadoop, Elastic Search
- Veri Bilimi - Data Science, Big Data, Python, R
- Makine Öğrenmesi, Yapay Zeka, Robotik

Yazılım

- Masaüstü - Desktop, Visual Studio, .NET, DLL, Component, QT, Linux Sistem
- Web - İnternet/Intranet, Backend/Frontend/Full Stack , .NET, Java, NodeJS, PHP, HTML, CSS, Javascript, Bootstrap, Angular, React, Vue
- Mobil - Android/iOS, Android Studio, Java, iOS Objective C, Swift, Cross Platform
- Sistem Programcılığı, DLL, Driver
- Gömülü Sistemler - Embedded, C/C++, Assembly, ARM, Arduino, RaspberryPi
- Sunucu Sistemleri, Web sunucu, Mail Sunucu, FTP Sunucu, Dosya Sunucu, Cloud Sistemler, Üyelik Tabanlı Sistemler, AWS, Microsoft Azure, Google Cloud
- Diller, Frameworkler, Geliştirme Ortamları

- Yazılım Geliştirme Yaşam Döngüsü - SDLC
 - Planlama, Fizibilite
 - Gereklilikler, Analiz
 - Tasarım ve Prototipleme
 - Yazılım Geliştirme, Kodlama
 - Yazılım Testi
 - Uygulama, kurulum, entegrasyon
 - İşletme (operasyon) ve Bakım

<https://www.clouddefense.ai/system-development-life-cycle/>

<https://www.guru99.com/software-development-life-cycle-tutorial.html>

NOTLAR

- Haftalik stajyer toplanti
- SaaS rotalama servisi
- SaaS gis cloud
- SaaS/mobil openroute ile navigasyon
- Projeleri birer sayfa tanimlama
- auto kiři sayma
- auto araç sayma
- auto real time plaka okuma, ürün

ÖRNEK PROJELER

Önemli Not: Bu projeler chatgpt ile haftalık plan halinde oluşturulmuştur. Projelere başlamadan önce haftalık planı gözden geçirip eksiklerini gidererek zenginleştiriniz. Staj sonunda projenin githuba yüklenmesi, sunum ile çalışırlığının gösterilmesi gerekmektedir.

Temel Bir Web Uygulaması Geliştirme - Haftalık Plan

Proje Tanımı

Basit bir web uygulaması geliştirme. Kullanıcıların veri girişi yapabileceği, verileri görüntüleyebileceği ve saklayabileceği temel bir web uygulaması.

Kullanılacak Teknolojiler

- HTML
- CSS
- JavaScript
- Önyüz Framework (React, Angular veya Vue.js)
- Backend Framework (Node.js/Express, Django veya Flask)
- Veritabanı (MySQL, PostgreSQL veya MongoDB)
- Hosting (Heroku, Netlify veya başka bir servis)

Haftalık Plan

1. Hafta: HTML ve CSS

Gün 1: HTML Temelleri

- Kaynaklar:
 - HTML Tutorial - W3Schools
 - [MDN HTML Guide](#)
- Pratik Çalışma:
 - HTML yapı taşları (etiketler, elementler, atributlar).
 - Basit bir HTML sayfası oluşturma.

Gün 2: HTML Formlar ve Tablolar

- Kaynaklar:
 - HTML Forms - W3Schools
 - HTML Tables - W3Schools
- Pratik Çalışma:
 - Form elemanları ve tablolar oluşturma.

Gün 3: CSS Temelleri

- Kaynaklar:
 - CSS Tutorial - W3Schools
 - [MDN CSS Guide](#)
- Pratik Çalışma:
 - CSS seçiciler, özellikler, ve değerler.
 - Basit stil uygulamaları yapma.

Gün 4: CSS Düzeni ve Flexbox

- Kaynaklar:
 - CSS Flexbox - W3Schools
 - [CSS Grid - MDN](#)
- Pratik Çalışma:
 - Flexbox ve Grid layout kullanarak sayfa düzeni oluşturma.

Gün 5: Proje Çalışması

- HTML ve CSS kullanarak basit bir web sayfası tasarlama.

2. Hafta: JavaScript ve Önyüz Framework

Gün 1: JavaScript Temelleri

- Kaynaklar:
 - JavaScript Tutorial - W3Schools
 - [MDN JavaScript Guide](#)
- Pratik Çalışma:
 - JavaScript temelleri (değişkenler, veri tipleri, döngüler, koşullar).

Gün 2: JavaScript DOM Manipülasyonu

- Kaynaklar:
 - JavaScript HTML DOM - W3Schools
 - [DOM Manipulation - MDN](#)
- Pratik Çalışma:
 - DOM elementi seçme ve değiştirme.

Gün 3: JavaScript Olay Yönetimi

- Kaynaklar:
 - JavaScript Events - W3Schools
 - [Handling Events - MDN](#)
- Pratik Çalışma:
 - Buton ve form olaylarını yönetme.

Gün 4: Önyüz Framework Tanıtımı (React)

- Kaynaklar:
 - React Tutorial - W3Schools
 - React Documentation
- Pratik Çalışma:
 - React bileşenlerini ve temel konseptleri öğrenme.

Gün 5: React ile Basit Uygulama

- Pratik Çalışma:
 - React kullanarak basit bir uygulama geliştirme.

3. Hafta: Backend ve Veritabanı

Gün 1: Node.js ve Express Tanıtımı

- Kaynaklar:
 - Node.js Tutorial - W3Schools
 - Express.js Guide
- Pratik Çalışma:
 - Basit bir Node.js ve Express sunucusu kurma.

Gün 2: RESTful API Oluşturma

- Kaynaklar:
 - RESTful API with Express
- Pratik Çalışma:
 - CRUD işlemleri için RESTful API geliştirme.

Gün 3: Veritabanı Temelleri ve SQL

- Kaynaklar:
 - SQL Tutorial - W3Schools
 - [MDN SQL Guide](#)
- Pratik Çalışma:
 - Veritabanı tabloları oluşturma ve temel SQL sorguları yazma.

Gün 4: Veritabanı Bağlantısı ve ORM Kullanımı

- Kaynaklar:
 - [Sequelize Documentation](#)
 - [Mongoose Documentation](#)
- Pratik Çalışma:
 - Express uygulamasını veritabanına bağlama ve ORM kullanma.

Gün 5: Proje Çalışması

- Node.js ve Express ile veritabanı bağlantılı bir API geliştirme.

4. Hafta: Uygulama Geliştirme ve Hosting

Gün 1: Önyüz ve Backend Entegrasyonu

- Pratik Çalışma:
 - React uygulamasını Node.js API ile entegre etme.

Gün 2: Kullanıcı Kimlik Doğrulama ve Yetkilendirme

- Kaynaklar:
 - JWT Authentication
 - Passport.js Documentation
- Pratik Çalışma:
 - Kullanıcı kimlik doğrulama ve yetkilendirme ekleme.

Gün 3: Uygulama Testi ve Hata Ayıklama

- Kaynaklar:
 - [Jest Documentation](#)
 - [Mocha Documentation](#)
- Pratik Çalışma:
 - Uygulama testleri yazma ve hata ayıklama.

Gün 4: Uygulama Hosting ve Dağıtım

- Kaynaklar:
 - Deploying Node.js App on Heroku
 - Deploying React App on Netlify
- Pratik Çalışma:
 - Uygulamayı Heroku veya Netlify gibi bir platformda barındırma.

Gün 5: Proje Çalışması ve Sunum

- Uygulamanın son kontrolleri ve sunumu için hazırlık.

Bu plan, stajyerlerin web uygulama geliştirme sürecini adım adım öğrenmelerini sağlayacak şekilde düzenlenmiştir. Haftalık plan, teorik bilgi ile pratik çalışmaları birleştirerek, stajyerlerin projede deneyim kazanmalarını sağlamayı amaçlamaktadır.

Rotalama Servisi Geliştirme Yol Haritası

Proje Tanımı

Bu proje, kullanıcıdan A ve B noktalarını alarak, bu noktalar arasında bir rotalama güzergahı çıkarmayı amaçlamaktadır. Kullanıcıya rota bilgisi yazılı olarak veya harita üzerinde gösterilecektir.

Kullanılacak Teknolojiler

- HTML: Web sayfası yapısının oluşturulması.
- CSS: Web sayfasının stil ve düzenlemelerinin yapılması.
- JavaScript: Dinamik ve etkileşimli özelliklerin eklenmesi.
- Önyüz Framework: React, Angular veya Vue.js.
- Harita Önyüz Framework: Leaflet, OpenLayers veya Mapbox GL JS.
- Harita Servisleri: Google Maps API, OpenStreetMap API.
- Arka Plan Servisi: Node.js, Python (Django/Flask) veya Ruby on Rails.
- Mekansal Veritabanı: PostGIS veya MongoDB.
- Coğrafi Yol Verisi: Shapefile, GeoJSON veya WMS/WFS servisleri.

Haftalık Plan

1. Hafta: HTML, CSS ve Temel JavaScript

- Gün 1: HTML Temelleri
 - W3Schools HTML Tutorial
 - [MDN HTML Guide](#)
 - Pratik: Basit bir web sayfası oluşturma.
- Gün 2: CSS Temelleri
 - W3Schools CSS Tutorial
 - [MDN CSS Guide](#)
 - Pratik: Web sayfasını stilize etme.
- Gün 3: JavaScript Temelleri
 - W3Schools JavaScript Tutorial
 - [MDN JavaScript Guide](#)
 - Pratik: Basit bir interaktif uygulama oluşturma.
- Gün 4: DOM Manipülasyonu
 - JavaScript DOM Manipulation
 - Pratik: Kullanıcıdan A ve B noktalarını almak için form oluşturma.
- Gün 5: Proje Çalışması
 - HTML, CSS ve JavaScript ile kullanıcı arayüzü oluşturma.
 - Kullanıcıdan veri alacak basit bir form oluşturma.

2. Hafta: Önyüz Framework ve Arka Plan Servisi

- Gün 1: Önyüz Framework Seçimi ve Kurulumu

- React: React Official Tutorial
- Angular: Angular Official Documentation
- Vue.js: Vue.js Official Guide
- Pratik: Seçilen framework'ü kurma ve basit bir bileşen oluşturma.
- Gün 2: Önyüz Geliştirme
 - Pratik: Kullanıcı arayüzünü önyüz framework'ü ile geliştirme.
- Gün 3: Arka Plan Servisi Kurulumu
 - Node.js: Node.js Official Documentation
 - Django: [Django Official Documentation](#)
 - Flask: Flask Official Documentation
 - Pratik: Basit bir API oluşturma.
- Gün 4: Veritabanı Kurulumu
 - PostGIS: PostGIS Documentation
 - MongoDB: [MongoDB Documentation](#)
 - Pratik: Mekansal veritabanı kurulumu ve veri yapılarının oluşturulması.
- Gün 5: Proje Çalışması
 - Kullanıcıdan alınan verilerin arka plan servisine gönderilmesi.
 - Arka plan servisi ile basit veri işlemleri.

3. Hafta: Harita ve Mekansal Veritabanı İşleri

- Gün 1: Harita Önyüz Framework Seçimi ve Kurulumu
 - Leaflet: [Leaflet Documentation](#)
 - OpenLayers: OpenLayers Documentation
 - Mapbox GL JS: Mapbox GL JS Documentation
 - Pratik: Seçilen harita framework'ü ile basit bir harita oluşturma.
- Gün 2: Harita Entegrasyonu
 - Pratik: Kullanıcı arayüzüne harita ekleme ve A ve B noktalarını gösterme.
- Gün 3: Mekansal Veritabanı ve Coğrafi Veriler
 - GeoJSON: [GeoJSON Format](#)
 - PostGIS: PostGIS Tutorials
 - Pratik: Mekansal veritabanı ile rota hesaplama.
- Gün 4: Harita Üzerinde Rota Görselleştirme
 - Pratik: A ve B noktaları arasındaki rotayı harita üzerinde gösterme.
- Gün 5: Proje Çalışması
 - Harita üzerinde kullanıcı rotasını gösteren uygulama geliştirme.

4. Hafta: Uygulama Geliştirme ve Test

- Gün 1: Kullanıcı Arayüzü İyileştirme
 - Pratik: Kullanıcı deneyimini artırmak için arayüz iyileştirmeleri.
- Gün 2: Performans Optimizasyonları
 - Web Performance Optimization
 - Pratik: Uygulama performansını artırma teknikleri.

- G n 3: G venlik  nlemleri
 - OWASP Web Security Testing Guide
 - Pratik: Uygulama g venlik  nlemleri ve testler.
- G n 4: Test ve Hata Ayıklama
 - Pratik: Uygulamanın test edilmesi ve hata ayıklama.
- G n 5: Proje  alışması
 - T m bileşenlerin entegrasyonu ve son testlerin yapılması.

5. Hafta: Yaygınlaştırma ve Bakım

- G n 1: Hosting ve Yaygınlaştırma
 - [Heroku](#)
 - [Netlify](#)
 - Pratik: Uygulamanın yaygınlaştırılması.
- G n 2: Kullanıcı Geri Bildirimleri
 - Pratik: Kullanıcı geri bildirimlerini toplama ve deęerlendirme.
- G n 3: Uygulama Bakımı
 - Pratik: Uygulamanın bakım s re leri.
- G n 4: Ek  zellikler Geliştirme
 - Pratik: Kullanıcı geri bildirimlerine g re yeni  zellikler geliştirme.
- G n 5: Proje Sunumu ve Deęerlendirme
 - Pratik: Projenin sunumu ve genel deęerlendirme.

Bu plan, stajyerlerin projeyi adım adım tamamlamalarına ve her hafta belirli konuları  ğrenmelerine yardımcı olacak şekilde d zenlenmiştir. Haftalık plan, teorik bilgi ile pratik  alışmaları birleştirerek, stajyerlerin projede deneyim kazanmalarını saęlamayı ama lamaktadır.

SaaS GIS Cloud Projesi - Haftalık Plan

Proje Tanımı

Web üzerinde çalışan basit bir GIS (Coğrafi Bilgi Sistemi) hizmeti geliştirilmesi. Bu hizmet, kullanıcıların verilerinin mekansal olarak oluşturulmasını ve görselleştirilmesini sağlayacak bir web hizmeti sunmayı amaçlamaktadır.

Kullanılacak Teknolojiler

- HTML: Web sayfası yapısının oluşturulması.
- CSS: Web sayfasının stil ve düzenlemelerinin yapılması.
- JavaScript: Dinamik ve etkileşimli özelliklerin eklenmesi.
- Önyüz Framework: React, Angular veya Vue.js gibi bir framework kullanılabilir.
- Harita Önyüz Framework: Leaflet, OpenLayers veya Mapbox GL JS gibi bir harita framework'ü.
- Harita Servisleri: Google Maps API, OpenStreetMap veya diğer harita servisleri.
- Arka Plan: Node.js, Python (Django/Flask) veya Ruby on Rails gibi bir arka uç teknolojisi.
- Mekansal Veritabanı: PostGIS veya MongoDB gibi bir mekansal veritabanı.
- Coğrafi Veriler: Shapefile, GeoJSON veya WMS/WFS servisleri.

Haftalık Plan

1. Hafta: HTML, CSS

Gün 1: HTML Temelleri

- Kaynaklar:
 - W3Schools HTML Tutorial
 - [MDN HTML Guide](#)
- Pratik Çalışma:
 - Temel bir web sayfası oluşturma.

Gün 2: CSS Temelleri

- Kaynaklar:
 - W3Schools CSS Tutorial
 - [MDN CSS Guide](#)
- Pratik Çalışma:
 - Web sayfasını stilize etme.

Gün 3: HTML ve CSS İleri Seviye Konular

- Kaynaklar:
 - CSS Flexbox Guide
 - CSS Grid Layout
- Pratik Çalışma:
 - Flexbox ve Grid ile düzen oluşturma.

Gün 4: Responsive Tasarım

- Kaynaklar:
 - Responsive Web Design Basics
- Pratik Çalışma:
 - Mobil uyumlu bir web sayfası oluşturma.

Gün 5: Proje Çalışması

- HTML ve CSS kullanarak kullanıcı arayüzü oluşturma.
- Form elemanları ve stil düzenlemeleri.

2. Hafta: JavaScript, Önyüz Framework, Arkaplan, Veritabanı, Hosting, Basit Web Uygulama

Gün 1: JavaScript Temelleri

- Kaynaklar:
 - W3Schools JavaScript Tutorial
 - [MDN JavaScript Guide](#)
- Pratik Çalışma:
 - Basit interaktif bir uygulama oluşturma.

Gün 2: Önyüz Framework Seçimi ve Kurulumu

- Kaynaklar:
 - React: React Official Tutorial
 - Angular: Angular Official Documentation
 - Vue.js: Vue.js Official Guide
- Pratik Çalışma:
 - Seçilen framework'ü kurma ve basit bir bileşen oluşturma.

Gün 3: Arkaplan Teknolojisi Kurulumu

- Kaynaklar:
 - Node.js: Node.js Official Documentation
 - Django: [Django Official Documentation](#)
 - Flask: Flask Official Documentation
- Pratik Çalışma:

- Basit bir API oluşturma.

Gün 4: Veritabanı Kurulumu

- Kaynaklar:
 - PostGIS: [PostGIS Documentation](#)
 - MongoDB: [MongoDB Documentation](#)
- Pratik Çalışma:
 - Mekansal veritabanı kurulumu ve veri yapılarının oluşturulması.

Gün 5: Hosting ve Basit Web Uygulama

- Kaynaklar:
 - [Heroku](#)
 - [Netlify](#)
- Pratik Çalışma:
 - Basit bir web uygulamasının hosting servisine dağıtılması.

3. Hafta: Harita ve Mekansal Veritabanı İşleri

Gün 1: Harita Önyüz Framework Seçimi ve Kurulumu

- Kaynaklar:
 - Leaflet: [Leaflet Documentation](#)
 - OpenLayers: [OpenLayers Documentation](#)
 - Mapbox GL JS: [Mapbox GL JS Documentation](#)
- Pratik Çalışma:
 - Seçilen harita framework'ü ile basit bir harita oluşturma.

Gün 2: Harita Entegrasyonu

- Pratik Çalışma:
 - Kullanıcı arayüzüne harita ekleme ve kullanıcı konumlarını gösterme.

Gün 3: Mekansal Veritabanı ve Coğrafi Veriler

- Kaynaklar:
 - GeoJSON: [GeoJSON Format](#)
 - PostGIS: [PostGIS Tutorials](#)
- Pratik Çalışma:
 - Mekansal veritabanı ile veri yönetimi.

Gün 4: Harita Üzerinde Veri Görselleştirme

- Pratik Çalışma:

- Kullanıcı verilerini harita üzerinde görselleştirme.

Gün 5: Proje Çalışması

- Harita ve mekansal veritabanı entegrasyonu ile uygulama geliştirme.

4. Hafta ve Devamı: Uygulama Geliştirme ve Test

Gün 1: Kullanıcı Arayüzü İyileştirme

- Pratik Çalışma:
 - Kullanıcı deneyimini artırmak için arayüz iyileştirmeleri.

Gün 2: Performans Optimizasyonları

- Kaynaklar:
 - Web Performance Optimization
- Pratik Çalışma:
 - Uygulama performansını artırma teknikleri.

Gün 3: Güvenlik Önlemleri

- Kaynaklar:
 - OWASP Web Security Testing Guide
- Pratik Çalışma:
 - Uygulama güvenlik önlemleri ve testler.

Gün 4: Test ve Hata Ayıklama

- Pratik Çalışma:
 - Uygulamanın test edilmesi ve hata ayıklama.

Gün 5: Proje Sunumu ve Değerlendirme

- Pratik Çalışma:
 - Projenin sunumu ve genel değerlendirme.

Bu plan, stajyerlerin projeyi adım adım tamamlamalarına ve her hafta belirli konuları öğrenmelerine yardımcı olacak şekilde düzenlenmiştir. Haftalık plan, teorik bilgi ile pratik çalışmaları birleştirerek, stajyerlerin projede deneyim kazanmalarını sağlamayı amaçlamaktadır.

Mobil Navigasyon Openroute Projesi - Haftalık Plan

Proje Tanımı

Bu proje, mobil cihazlarda A noktasından B noktasına gidecek şekilde bir rotalama güzergahı çıkarmayı amaçlamaktadır. Kullanıcıdan A ve B noktaları alınacak ve arka planda çalışan servis sorgusu ile kullanıcıya gideceği rota gösterilecek ve navigasyon yapılacaktır.

Kullanılacak Teknolojiler

- Mobil Programlama: React Native, Flutter, Swift, Kotlin
- Harita Servisleri: Google Maps API, Mapbox, OpenStreetMap
- Arkaplan: Node.js, Python (Django/Flask), Firebase
- Mekansal Veritabanı: PostGIS, MongoDB
- Coğrafi Yol Verisi: OpenRouteService API, GeoJSON

Haftalık Plan

1. Hafta: Mobil Programlama

Gün 1: Mobil Programlama Temelleri

- Kaynaklar:
 - React Native: React Native Documentation
 - Flutter: Flutter Documentation
 - Swift: Swift Programming Language Guide
 - Kotlin: Kotlin Documentation
- Pratik Çalışma:
 - Basit bir "Hello World" mobil uygulaması oluşturma.

Gün 2: Kullanıcı Arayüzü Tasarımı

- Kaynaklar:
 - React Native: React Native Styling
 - Flutter: Flutter Layouts
 - Swift: [SwiftUI Tutorials](#)
 - Kotlin: [Android Layouts](#)
- Pratik Çalışma:
 - Kullanıcı arayüzü bileşenleri ve stil düzenlemeleri.

Gün 3: Kullanıcı Girdileri ve Formlar

- Kaynaklar:
 - React Native: React Native Forms

- Flutter: Flutter Forms
- Swift: [Swift Forms](#)
- Kotlin: [Android Forms](#)
- Pratik Çalışma:
 - Kullanıcıdan A ve B noktalarını almak için form oluşturma.

Gün 4: Mobil Uygulama Navigasyonu

- Kaynaklar:
 - React Native: [React Navigation](#)
 - Flutter: Flutter Navigation
 - Swift: [SwiftUI Navigation](#)
 - Kotlin: [Android Navigation](#)
- Pratik Çalışma:
 - Uygulama içinde sayfalar arası geçiş yapma.

Gün 5: Proje Çalışması

- Mobil uygulama temelleri ile kullanıcı arayüzü ve navigasyon bileşenlerini oluşturma.

2. Hafta: Mobil Programlama, Arkaplan, Veritabanı, Hosting, Basit Mobil Uygulama

Gün 1: Arkaplan Teknolojisi Kurulumu

- Kaynaklar:
 - Node.js: Node.js Official Documentation
 - Django: [Django Official Documentation](#)
 - Flask: Flask Official Documentation
 - Firebase: Firebase Documentation
- Pratik Çalışma:
 - Basit bir API oluşturma ve veri alıp gönderme.

Gün 2: Veritabanı Kurulumu

- Kaynaklar:
 - PostGIS: PostGIS Documentation
 - MongoDB: [MongoDB Documentation](#)
 - Firebase Firestore: Firestore Documentation
- Pratik Çalışma:
 - Mekansal veritabanı kurulumu ve veri yapılarının oluşturulması.

Gün 3: Basit Mobil Uygulama Entegrasyonu

- Kaynaklar:

- REST API: [REST API Tutorial](#)
- GraphQL: GraphQL Documentation
- Pratik Çalışma:
 - Mobil uygulama ile arka plan servisi arasında veri alışverişi.

Gün 4: Hosting ve Yaygınlaştırma

- Kaynaklar:
 - Heroku: Heroku Documentation
 - Firebase Hosting: Firebase Hosting
- Pratik Çalışma:
 - Basit bir arka plan servisinin yaygınlaştırılması.

Gün 5: Proje Çalışması

- Mobil uygulama ile arka plan servisi ve veritabanı entegrasyonunu tamamlama.

3. Hafta: Harita, Mekansal Veritabanı, OpenRoute

Gün 1: Harita Servisleri Kurulumu

- Kaynaklar:
 - Google Maps: Google Maps API Documentation
 - Mapbox: Mapbox Documentation
 - OpenStreetMap: OpenStreetMap API
- Pratik Çalışma:
 - Harita servislerini mobil uygulamaya entegre etme.

Gün 2: Harita Üzerinde Noktaları Gösterme

- Kaynaklar:
 - GeoJSON: [GeoJSON Format](#)
- Pratik Çalışma:
 - Kullanıcıdan alınan A ve B noktalarını harita üzerinde gösterme.

Gün 3: OpenRouteService API Entegrasyonu

- Kaynaklar:
 - OpenRouteService: OpenRouteService Documentation
- Pratik Çalışma:
 - OpenRouteService API ile rota hesaplama.

Gün 4: Rota Görselleştirme

- Pratik Çalışma:
 - Hesaplanan rotayı harita üzerinde görselleştirme.

Gün 5: Proje Çalışması

- Harita ve rota hesaplama işlevselliğini mobil uygulamaya tam entegrasyon.

4. Hafta ve Devamı: Uygulama Geliştirme ve Test

Gün 1: Kullanıcı Arayüzü İyileştirme

- Pratik Çalışma:
 - Kullanıcı deneyimini artırmak için arayüz iyileştirmeleri.

Gün 2: Performans Optimizasyonları

- Kaynaklar:
 - Mobile App Performance Optimization
- Pratik Çalışma:
 - Uygulama performansını artırma teknikleri.

Gün 3: Güvenlik Önlemleri

- Kaynaklar:
 - OWASP Mobile Security Testing Guide
- Pratik Çalışma:
 - Uygulama güvenlik önlemleri ve testler.

Gün 4: Test ve Hata Ayıklama

- Kaynaklar:
 - [Automated Testing](#)
 - [iOS Testing](#)
- Pratik Çalışma:
 - Uygulamanın test edilmesi ve hata ayıklama.

Gün 5: Proje Sunumu ve Değerlendirme

- Pratik Çalışma:
 - Projenin sunumu ve genel değerlendirme.

Bu plan, stajyerlerin projeyi adım adım tamamlamalarına ve her hafta belirli konuları öğrenmelerine yardımcı olacak şekilde düzenlenmiştir. Haftalık plan, teorik bilgi ile pratik çalışmaları birleştirerek, stajyerlerin projede deneyim kazanmalarını sağlamayı amaçlamaktadır.

Kişi Sayma Uygulaması

Proje Tanımı

Kameradan alınan video görüntüleri üzerinden kişi sayma uygulaması geliştirme. Hem gerçek zamanlı hem de offline çalışabilir.

Kullanılacak Teknolojiler

- Python
- Jetson
- Yapay Zeka (YZ) Temelleri
- Model Eğitimi

1. Hafta: Python Temelleri

Hedef: Python dilinde temel bilgi ve beceriler kazanmak.

Gün 1: Python'a Giriş

- Python Kurulum ve IDE:
 - [Python İndir](#)
 - [VSCode İndir](#)
 - [PyCharm İndir](#)
- Temel Python Bilgileri:
 - [Python Belgeseli](#)
 - [Python Başlangıç Kılavuzu](#)

Gün 2: Python Temel Bilgileri

- Python Temel Bilgiler:
 - Python Koşul İfadeleri ve Döngüler
 - Python Veri Türleri

Gün 3: Fonksiyonlar ve Modüller

- Fonksiyonlar ve Modüller
- Python İstisna Yönetimi

Gün 4: Veri Manipülasyonu

- NumPy Kılavuzu
- pandas Kılavuzu

Gün 5: Veri Görselleştirme

- Matplotlib ile Görselleştirme
 - Ödev: Python Veri Analizi Projesi Örneği
-

2. Hafta: Yapay Zeka Temelleri ve Model Eğitimi

Hedef: Yapay Zeka ve makine öğrenimi temellerini öğrenmek.

Gün 1: Yapay Zeka ve Makine Öğrenimi Temelleri

- [Yapay Zeka ve Makine Öğrenimi Temelleri](#)
- Makine Öğrenimi Algoritmaları

Gün 2: Veri Setleri ve Özellik Mühendisliği

- Veri Ön İşleme
- Özellik Mühendisliği

Gün 3: Model Eğitimi

- Model Eğitimi ve Değerlendirme

Gün 4: Model Değerlendirme ve Hiperparametre Ayarları

- Hiperparametre Ayarları

Gün 5: Model Eğitimi Uygulaması

- Ödev: Makine Öğrenimi Projeleri Örneği
-

3. Hafta: NVIDIA Jetson ve Gömülü Sistemler

Hedef: NVIDIA Jetson platformunu öğrenmek ve model geliştirmek.

Gün 1: NVIDIA Jetson'a Giriş

- [Jetson Başlangıç Kılavuzu](#)
- [Jetson Nano Setup](#)

Gün 2: Model Dönüştürme

- [TensorRT ile Model Optimizasyonu](#)
- [ONNX Formatı](#)

Gün 3: Jetson ve Video İşleme

- GStreamer ile Video İşleme

Gün 4: Jetson Üzerinde Video Akışı

- [Jetson ile Video Akışı](#)

Gün 5: Jetson Uygulama Geliştirme ve Ödev

- [Jetson Üzerinde Basit Model Geliştirme Örneği](#)
-

4. Hafta ve Sonrası: Kişi Sayma Uygulaması

Hedef: Kişi sayma uygulamasını geliştirmek ve test etmek.

Gün 1: Proje Planlaması ve Gereksinimler

- Proje Planlama Rehberi
- Yapay Zeka Proje Yönetimi

Gün 2: Model Seçimi

- Kişi Sayma İçin YOLO Modeli

Gün 3: Model Eğitim ve Değerlendirme

- SSD Modeli ile Kişi Tespiti

Gün 4: Uygulama Geliştirme

- Python ve OpenCV ile Video İşleme
- [Jetson ile Gerçek Zamanlı Video İşleme](#)

Gün 5: Test ve Optimizasyon

- Uygulama Testi ve Optimizasyon
- Performans İyileştirme Rehberi

Gün 6-10: Dokümantasyon ve Sunum

- Teknik Dokümantasyon Rehberi
- [Proje Sunumu İçin İpuçları](#)

Sonraki Haftalar:

- Uygulama Bakımı ve Geliştirme

Kişi Tanıma Uygulaması - Haftalık Plan

Proje Tanımı

Kameradan alınan video görüntüleri üzerinden kişi tanıma uygulaması geliştirme. Uygulama hem gerçek zamanlı hem de offline çalışabilir.

Kullanılacak Teknolojiler

- Python
- NVIDIA Jetson
- Yapay Zeka (YZ) Temelleri
- Model Eğitimi

Haftalık Plan

1. Hafta: Python ve Yapay Zeka Temelleri

Gün 1: Python Programlama Temelleri

- Kaynaklar:
 - W3Schools Python Tutorial
 - [Real Python Tutorials](#)
- Pratik Çalışma:
 - Python kurulum ve temel programlama kavramları (değişkenler, veri tipleri, döngüler, koşullar).

Gün 2: Python Veri İşleme ve Kütüphaneler

- Kaynaklar:
 - Pandas Documentation
 - NumPy Documentation
- Pratik Çalışma:
 - Pandas ve NumPy kullanarak veri işleme.

Gün 3: Yapay Zeka ve Makine Öğrenimi Temelleri

- Kaynaklar:
 - [Coursera Machine Learning Course](#)
 - Fast.ai Introduction to Machine Learning
- Pratik Çalışma:
 - Temel makine öğrenimi kavramları ve algoritmaları.

Gün 4: Python ile YZ Kütüphaneleri

- Kaynaklar:

- Scikit-Learn Documentation
- [TensorFlow Documentation](#)
- PyTorch Documentation
- Pratik Çalışma:
 - Basit makine öğrenimi modelleri oluşturma ve eğitme.

Gün 5: Proje Çalışması

- Python ve YZ temelleriyle ilgili öğrendiklerini pekiştirmek için küçük projeler yapma.

2. Hafta: NVIDIA Jetson ve Model Eğitimi

Gün 1: NVIDIA Jetson Platformuna Giriş

- Kaynaklar:
 - [NVIDIA Jetson Getting Started Guide](#)
- Pratik Çalışma:
 - Jetson kurulumu ve temel konfigürasyonu.

Gün 2: Jetson ile Python ve YZ Çalıştırma

- Kaynaklar:
 - [Jetson AI Fundamentals](#)
- Pratik Çalışma:
 - Jetson üzerinde Python çalıştırma ve basit YZ modellerini çalıştırma.

Gün 3: Görüntü İşleme ve OpenCV

- Kaynaklar:
 - OpenCV Documentation
- Pratik Çalışma:
 - OpenCV kullanarak görüntü işleme temelleri.

Gün 4: Yüz Tanıma Modelleri ve Veri Toplama

- Kaynaklar:
 - Dlib Face Recognition
 - [FaceNet Paper](#)
- Pratik Çalışma:
 - Yüz tanıma modelleri hakkında bilgi edinme ve veri toplama.

Gün 5: Proje Çalışması

- Jetson üzerinde OpenCV ve yüz tanıma modelini çalıştırma.

3. Hafta: Model Eğitimi ve Entegrasyon

Gün 1: Model Eğitimi için Veri Hazırlama

- Kaynaklar:
 - Image Data Augmentation
- Pratik Çalışma:
 - Veri artırma ve veri seti hazırlama.

Gün 2: Yüz Tanıma Modeli Eğitimi

- Kaynaklar:
 - [Face Recognition with Keras](#)
- Pratik Çalışma:
 - Yüz tanıma modelinin eğitilmesi.

Gün 3: Modelin Değerlendirilmesi ve Optimizasyonu

- Kaynaklar:
 - Model Evaluation Techniques
- Pratik Çalışma:
 - Modelin doğruluk, kesinlik, geri çağırma gibi metriklerle değerlendirilmesi.

Gün 4: Modelin Jetson'a Dağıtımı

- Kaynaklar:
 - [Deploying Deep Learning Models on NVIDIA Jetson](#)
- Pratik Çalışma:
 - Eğitilen modelin Jetson'a dağıtılması ve optimizasyonu.

Gün 5: Proje Çalışması

- Modelin entegrasyonu ve Jetson üzerinde çalıştırılması.

4. Hafta ve Devamı: Uygulama Geliştirme ve Test

Gün 1: Gerçek Zamanlı Yüz Tanıma Uygulaması

- Pratik Çalışma:
 - Kamera ile gerçek zamanlı yüz tanıma uygulaması geliştirme.

Gün 2: Offline Yüz Tanıma Uygulaması

- Pratik Çalışma:

- Kayıtlı video veya görüntüler üzerinden offline yüz tanıma uygulaması geliştirme.

Gün 3: Performans Optimizasyonları

- Kaynaklar:
 - [Optimizing Deep Learning Models on Jetson](#)
- Pratik Çalışma:
 - Uygulamanın performansını artırma teknikleri.

Gün 4: Güvenlik ve Veri Gizliliği

- Kaynaklar:
 - OWASP Privacy Risks
- Pratik Çalışma:
 - Uygulama güvenlik önlemleri ve veri gizliliği konuları.

Gün 5: Test ve Hata Ayıklama

- Pratik Çalışma:
 - Uygulamanın test edilmesi ve hata ayıklama.

Devam Eden Günler: Proje Sunumu ve Değerlendirme

- Pratik Çalışma:
 - Projenin sunumu ve genel değerlendirme.

Bu plan, stajyerlerin projeyi adım adım tamamlamalarına ve her hafta belirli konuları öğrenmelerine yardımcı olacak şekilde düzenlenmiştir. Haftalık plan, teorik bilgi ile pratik çalışmaları birleştirerek, stajyerlerin projede deneyim kazanmalarını sağlamayı amaçlamaktadır.

Araç Sayma Uygulaması - Haftalık Plan

Proje Tanımı

Kameradan alınan video görüntüleri üzerinden araç sayma uygulaması geliştirme. Uygulama hem gerçek zamanlı hem de offline çalışabilir.

Kullanılacak Teknolojiler

- Python
- NVIDIA Jetson
- Yapay Zeka (YZ) Temelleri
- Model Eğitimi

Haftalık Plan

1. Hafta: Python ve Yapay Zeka Temelleri

Gün 1: Python Programlama Temelleri

- Kaynaklar:
 - W3Schools Python Tutorial
 - [Real Python Tutorials](#)
- Pratik Çalışma:
 - Python kurulum ve temel programlama kavramları (değişkenler, veri tipleri, döngüler, koşullar).

Gün 2: Python Veri İşleme ve Kütüphaneler

- Kaynaklar:
 - Pandas Documentation
 - NumPy Documentation
- Pratik Çalışma:
 - Pandas ve NumPy kullanarak veri işleme.

Gün 3: Yapay Zeka ve Makine Öğrenimi Temelleri

- Kaynaklar:
 - [Coursera Machine Learning Course](#)
 - Fast.ai Introduction to Machine Learning
- Pratik Çalışma:
 - Temel makine öğrenimi kavramları ve algoritmaları.

Gün 4: Python ile YZ Kütüphaneleri

- Kaynaklar:

- Scikit-Learn Documentation
- [TensorFlow Documentation](#)
- PyTorch Documentation
- Pratik Çalışma:
 - Basit makine öğrenimi modelleri oluşturma ve eğitme.

Gün 5: Proje Çalışması

- Python ve YZ temelleriyle ilgili öğrendiklerini pekiştirmek için küçük projeler yapma.

2. Hafta: NVIDIA Jetson ve Model Eğitimi

Gün 1: NVIDIA Jetson Platformuna Giriş

- Kaynaklar:
 - [NVIDIA Jetson Getting Started Guide](#)
- Pratik Çalışma:
 - Jetson kurulumu ve temel konfigürasyonu.

Gün 2: Jetson ile Python ve YZ Çalıştırma

- Kaynaklar:
 - [Jetson AI Fundamentals](#)
- Pratik Çalışma:
 - Jetson üzerinde Python çalıştırma ve basit YZ modellerini çalıştırma.

Gün 3: Görüntü İşleme ve OpenCV

- Kaynaklar:
 - OpenCV Documentation
- Pratik Çalışma:
 - OpenCV kullanarak görüntü işleme temelleri.

Gün 4: Araç Tanıma Modelleri ve Veri Toplama

- Kaynaklar:
 - [YOLOv4 for Vehicle Detection](#)
 - [TensorFlow Object Detection API](#)
- Pratik Çalışma:
 - Araç tanıma modelleri hakkında bilgi edinme ve veri toplama.

Gün 5: Proje Çalışması

- Jetson üzerinde OpenCV ve araç tanıma modelini çalıştırma.

3. Hafta: Model Eğitimi ve Entegrasyon

Gün 1: Model Eğitimi için Veri Hazırlama

- Kaynaklar:
 - Image Data Augmentation
- Pratik Çalışma:
 - Veri artırma ve veri seti hazırlama.

Gün 2: Araç Tanıma Modeli Eğitimi

- Kaynaklar:
 - [Training YOLO on Custom Dataset](#)
- Pratik Çalışma:
 - Araç tanıma modelinin eğitilmesi.

Gün 3: Modelin Değerlendirilmesi ve Optimizasyonu

- Kaynaklar:
 - Model Evaluation Techniques
- Pratik Çalışma:
 - Modelin doğruluk, kesinlik, geri çağırma gibi metriklerle değerlendirilmesi.

Gün 4: Modelin Jetson'a Dağıtımı

- Kaynaklar:
 - [Deploying Deep Learning Models on NVIDIA Jetson](#)
- Pratik Çalışma:
 - Eğitilen modelin Jetson'a dağıtılması ve optimizasyonu.

Gün 5: Proje Çalışması

- Modelin entegrasyonu ve Jetson üzerinde çalıştırılması.

4. Hafta ve Devamı: Uygulama Geliştirme ve Test

Gün 1: Gerçek Zamanlı Araç Sayma Uygulaması

- Pratik Çalışma:
 - Kamera ile gerçek zamanlı araç sayma uygulaması geliştirme.

Gün 2: Offline Araç Sayma Uygulaması

- Pratik Çalışma:

- Kayıtlı video veya görüntüler üzerinden offline araç sayma uygulaması geliştirme.

Gün 3: Performans Optimizasyonları

- Kaynaklar:
 - [Optimizing Deep Learning Models on Jetson](#)
- Pratik Çalışma:
 - Uygulamanın performansını artırma teknikleri.

Gün 4: Güvenlik ve Veri Gizliliği

- Kaynaklar:
 - OWASP Privacy Risks
- Pratik Çalışma:
 - Uygulama güvenlik önlemleri ve veri gizliliği konuları.

Gün 5: Test ve Hata Ayıklama

- Pratik Çalışma:
 - Uygulamanın test edilmesi ve hata ayıklama.

Devam Eden Günler: Proje Sunumu ve Değerlendirme

- Pratik Çalışma:
 - Projenin sunumu ve genel değerlendirme.

Bu plan, stajyerlerin projeyi adım adım tamamlamalarına ve her hafta belirli konuları öğrenmelerine yardımcı olacak şekilde düzenlenmiştir. Haftalık plan, teorik bilgi ile pratik çalışmaları birleştirerek, stajyerlerin projede deneyim kazanmalarını sağlamayı amaçlamaktadır.

Plaka Okuma Uygulaması - Haftalık Plan

Proje Tanımı

Kameradan alınan video görüntüleri üzerinden araç plakalarını tanıma ve okuma uygulaması geliştirme. Uygulama hem gerçek zamanlı hem de offline çalışabilir.

Kullanılacak Teknolojiler

- Python
- NVIDIA Jetson
- Yapay Zeka (YZ) Temelleri
- Model Eğitimi

Haftalık Plan

1. Hafta: Python ve Yapay Zeka Temelleri

Gün 1: Python Programlama Temelleri

- Kaynaklar:
 - W3Schools Python Tutorial
 - [Real Python Tutorials](#)
- Pratik Çalışma:
 - Python kurulum ve temel programlama kavramları (değişkenler, veri tipleri, döngüler, koşullar).

Gün 2: Python Veri İşleme ve Kütüphaneler

- Kaynaklar:
 - Pandas Documentation
 - NumPy Documentation
- Pratik Çalışma:
 - Pandas ve NumPy kullanarak veri işleme.

Gün 3: Yapay Zeka ve Makine Öğrenimi Temelleri

- Kaynaklar:
 - [Coursera Machine Learning Course](#)
 - Fast.ai Introduction to Machine Learning
- Pratik Çalışma:
 - Temel makine öğrenimi kavramları ve algoritmaları.

Gün 4: Python ile YZ Kütüphaneleri

- Kaynaklar:

- Scikit-Learn Documentation
- [TensorFlow Documentation](#)
- PyTorch Documentation
- Pratik Çalışma:
 - Basit makine öğrenimi modelleri oluşturma ve eğitme.

Gün 5: Proje Çalışması

- Python ve YZ temelleriyle ilgili öğrendiklerini pekiştirmek için küçük projeler yapma.

2. Hafta: NVIDIA Jetson ve Model Eğitimi

Gün 1: NVIDIA Jetson Platformuna Giriş

- Kaynaklar:
 - [NVIDIA Jetson Getting Started Guide](#)
- Pratik Çalışma:
 - Jetson kurulumu ve temel konfigürasyonu.

Gün 2: Jetson ile Python ve YZ Çalıştırma

- Kaynaklar:
 - [Jetson AI Fundamentals](#)
- Pratik Çalışma:
 - Jetson üzerinde Python çalıştırma ve basit YZ modellerini çalıştırma.

Gün 3: Görüntü İşleme ve OpenCV

- Kaynaklar:
 - OpenCV Documentation
- Pratik Çalışma:
 - OpenCV kullanarak görüntü işleme temelleri.

Gün 4: Plaka Tanıma Modelleri ve Veri Toplama

- Kaynaklar:
 - [Anpr License Plate Recognition](#)
 - [YOLOv4 for License Plate Detection](#)
- Pratik Çalışma:
 - Plaka tanıma modelleri hakkında bilgi edinme ve veri toplama.

Gün 5: Proje Çalışması

- Jetson üzerinde OpenCV ve plaka tanıma modelini çalıştırma.

3. Hafta: Model Eğitimi ve Entegrasyon

Gün 1: Model Eğitimi için Veri Hazırlama

- Kaynaklar:
 - Image Data Augmentation
- Pratik Çalışma:
 - Veri artırma ve veri seti hazırlama.

Gün 2: Plaka Tanıma Modeli Eğitimi

- Kaynaklar:
 - [Training YOLO on Custom Dataset](#)
- Pratik Çalışma:
 - Plaka tanıma modelinin eğitilmesi.

Gün 3: Modelin Değerlendirilmesi ve Optimizasyonu

- Kaynaklar:
 - Model Evaluation Techniques
- Pratik Çalışma:
 - Modelin doğruluk, kesinlik, geri çağırma gibi metriklerle değerlendirilmesi.

Gün 4: Modelin Jetson'a Dağıtımı

- Kaynaklar:
 - [Deploying Deep Learning Models on NVIDIA Jetson](#)
- Pratik Çalışma:
 - Eğitilen modelin Jetson'a dağıtılması ve optimizasyonu.

Gün 5: Proje Çalışması

- Modelin entegrasyonu ve Jetson üzerinde çalıştırılması.

4. Hafta ve Devamı: Uygulama Geliştirme ve Test

Gün 1: Gerçek Zamanlı Plaka Okuma Uygulaması

- Pratik Çalışma:
 - Kamera ile gerçek zamanlı plaka okuma uygulaması geliştirme.

Gün 2: Offline Plaka Okuma Uygulaması

- Pratik Çalışma:

- Kayıtlı video veya görüntüler üzerinden offline plaka okuma uygulaması geliştirme.

Gün 3: Performans Optimizasyonları

- Kaynaklar:
 - [Optimizing Deep Learning Models on Jetson](#)
- Pratik Çalışma:
 - Uygulamanın performansını artırma teknikleri.

Gün 4: Güvenlik ve Veri Gizliliği

- Kaynaklar:
 - OWASP Privacy Risks
- Pratik Çalışma:
 - Uygulama güvenlik önlemleri ve veri gizliliği konuları.

Gün 5: Test ve Hata Ayıklama

- Pratik Çalışma:
 - Uygulamanın test edilmesi ve hata ayıklama.

Devam Eden Günler: Proje Sunumu ve Değerlendirme

- Pratik Çalışma:
 - Projenin sunumu ve genel değerlendirme.

Bu plan, stajyerlerin projeyi adım adım tamamlamalarına ve her hafta belirli konuları öğrenmelerine yardımcı olacak şekilde düzenlenmiştir. Haftalık plan, teorik bilgi ile pratik çalışmaları birleştirerek, stajyerlerin projede deneyim kazanmalarını sağlamayı amaçlamaktadır.

Taksi Çağırma Uygulaması

Uber benzeri bir uygulama yapmak için adımlar oldukça geniş bir kapsama sahiptir, çünkü bu tür bir uygulama hem mobil (iOS ve Android) hem de sunucu tarafında güçlü bir yapı gerektirir. Aşağıda adım adım bir Uber benzeri uygulamanın geliştirilmesi süreci bulunmaktadır

1. Fikir ve Proje Planlaması

- **Pazar Araştırması:** Uber'in ve diğer benzer uygulamaların pazardaki yeri, kullanıcı alışkanlıkları ve ihtiyacı belirlenmelidir.
- **Uygulama İş Modeli:** Ücretlendirme modeli, komisyonlar, kullanıcıya sunulan hizmetler (VIP, düşük maliyetli seçenekler) gibi unsurlar tanımlanmalıdır.
- **Özellikler Listesi:** Yolcu, sürücü ve admin panelleri için gerekli olan tüm özellikler detaylandırılmalıdır (örn. sürücü konum takibi, ödemeler, bildirimler).

2. Gerekli Teknolojilerin Belirlenmesi

- **Mobil Uygulama:**
 - iOS ve Android için mobil uygulama geliştirme araçları seçilmelidir. Native (Swift, Kotlin) ya da çapraz platform (React Native, Flutter) seçenekleri gözden geçirilmelidir.
- **Arka Uç (Backend):**
 - Sunucu tarafında Node.js, Python (Django/Flask), Ruby on Rails gibi teknolojiler kullanılabilir. Genişleyen bir kullanıcı tabanı için **AWS**, **Google Cloud Platform** veya **Microsoft Azure** gibi bulut servisleri tercih edilebilir.
 - **Veritabanı:** Kullanıcı, sürücü ve yolculuk bilgilerini depolamak için SQL (MySQL, PostgreSQL) veya NoSQL (MongoDB) veritabanları tercih edilebilir.
- **Gerçek Zamanlı İletişim:**
 - **WebSocket** veya **Firebase Real-Time Database** gibi çözümler, sürücü ve yolcu arasında gerçek zamanlı konum paylaşımı ve iletişim sağlamak için kullanılabilir.
- **Harita ve GPS Servisleri:**
 - **Google Maps API** ya da **Mapbox API** gibi harita servisleri ile kullanıcıların konumları izlenebilir ve yönlendirme yapılabilir.
- **Ödeme Entegrasyonu:**
 - **Stripe**, **PayPal**, veya **Braintree** gibi servisler ile kredi kartı ödemeleri entegre edilmelidir.

3. Mobil Uygulamanın Geliştirilmesi

- **Kullanıcı ve Sürücü Panelleri:** İki ayrı uygulama geliştirilebilir veya aynı uygulama içinde iki farklı giriş yapılabilir.
- **Harita Entegrasyonu:** Kullanıcı ve sürücünün haritada konumlarını görmeleri sağlanmalı ve rota çizme işlemleri yapılmalıdır.
- **Bildirimler:** Gerçek zamanlı bildirimler için **Push Notification** servisleri (örn. **Firebase Cloud Messaging** veya **Apple Push Notification** servisi) kullanılabilir.

4. Arka Uç (Backend) Geliştirilmesi

- **API Geliştirme:** Mobil uygulamanın sunucuya istek gönderebilmesi için RESTful API'lar ya da GraphQL yapıları oluşturulmalıdır.
- **Veri Yönetimi ve Depolama:** Sürücü ve yolculuk bilgileri, yolcu/sürücü geçmişi, ödemeler gibi veriler güvenli bir şekilde saklanmalıdır.
- **Ödeme İşlemleri:** Kullanıcıların kredi kartı bilgileri ve ödemeleri güvenli bir şekilde işlenmelidir. Ödeme işlemleri için PCI DSS (Payment Card Industry Data Security Standard) gibi güvenlik standartlarına uyulmalıdır.

5. Gerçek Zamanlı Takip ve Rota Hesaplama

- **Sürücü Konum Takibi:** Sürücülerin konumlarını gerçek zamanlı olarak kullanıcıya göstermek için GPS verileri kullanılır. Bu verilerin sunucuya hızlı ve güvenilir bir şekilde iletilmesi gereklidir.
- **Rota ve Mesafe Hesaplama:** Google Maps veya benzeri bir servis ile en kısa ya da en hızlı rotanın hesaplanması ve mesafeye göre ücretlendirme yapılmalıdır.

6. Güvenlik ve Kullanıcı Doğrulama

- **Kullanıcı ve Sürücü Doğrulama:** Telefon numarası, e-posta doğrulama veya sosyal medya hesaplarıyla giriş yapılmasını sağlayan özellikler eklenmelidir.
- **Güvenlik Önlemleri:** Kullanıcı ve sürücü güvenliğini artırmak için yolculuk geçmişi, araç bilgileri ve kullanıcı/sürücü yorumları gibi özellikler sunulabilir.

7. Test Süreci

- **Fonksiyonel Testler:** Uygulamanın temel işlevlerinin sorunsuz çalıştığını test etmek gerekir.
- **Kullanıcı Deneyimi Testleri:** Uygulamanın kullanıcı dostu olup olmadığını kontrol etmek için beta testler yapılmalıdır.
- **Yük ve Performans Testleri:** Uygulamanın yüksek kullanıcı trafiğinde nasıl performans gösterdiği ölçülmelidir.

8. Lansman ve Yayınlama

- **Mobil Mağazalara Yayınlama:** Uygulama App Store ve Google Play Store'a yüklenmelidir. Mağaza kuralları ve yönergeleri dikkatlice uygulanmalıdır.
- **Pazarlama ve Kullanıcı Kazanımı:** Sosyal medya, dijital pazarlama ve reklamlarla kullanıcı ve sürücülerin ilgisini çekmek için çalışmalar başlatılmalıdır.

9. Destek ve Güncellemeler

- **Kullanıcı ve Sürücü Destek Hattı:** Uygulama kullanıcılara hızlı ve etkili bir müşteri hizmeti sunmalıdır.
- **Sürekli Güncellemeler:** Uygulama performansını artırmak, hataları düzeltmek ve yeni özellikler eklemek için düzenli güncellemeler yapılmalıdır.

Bu adımlarla Uber benzeri bir uygulama geliştirme sürecine başlayabilirsiniz.

Uber benzeri bir uygulama için MVP (Minimum Viable Product) geliştirmenin maliyeti, tam özellikli bir uygulamaya göre daha düşük olacaktır. Ancak yine de belirli temel işlevler yerine getirilmelidir. MVP, uygulamanın en temel sürümüdür ve piyasaya hızlıca çıkıp kullanıcı geri bildirimleri almayı amaçlar. Bu sürümde yer alması gereken temel özellikler ve tahmini maliyetler şunlardır:

MVP'de Bulunması Gereken Temel Özellikler:

1. **Kullanıcı ve Sürücü Kaydı:** Kullanıcılar ve sürücüler için giriş/kayıt ekranları, e-posta veya sosyal medya ile giriş.
2. **Harita Entegrasyonu:** Google Maps veya Mapbox API ile yolculuklar için harita entegrasyonu, konum takibi ve rota belirleme.
3. **Yolculuk Talebi:** Kullanıcının yolculuk talebinde bulunabileceği ve sürücünün bu talepleri görebileceği bir sistem.
4. **Ödeme Entegrasyonu:** Kredi kartı veya PayPal gibi temel ödeme sistemlerinin entegre edilmesi.
5. **Temel Bildirimler:** Push bildirimleri ile sürücü/kullanıcı bilgilendirme.
6. **Yolculuk Geçmişi:** Kullanıcıların ve sürücülerin önceki yolculuklarını görebileceği bir geçmiş.
7. **Temel Yorum ve Puanlama Sistemi:** Kullanıcılar ve sürücüler için birbirlerini değerlendirme ve yorum bırakma özelliği.

MapLibre kullanarak navigasyon uygulaması, web/mobil

Creating a navigation app using MapLibre involves multiple steps, from setting up the basic map interface to implementing navigation features such as routing, geolocation, and turn-by-turn directions. Here's a step-by-step guide to get started:

1. Setup Your Development Environment

- Install Node.js for building a JavaScript-based web app.
- Choose your framework or library (e.g., React, Angular, or vanilla JavaScript).

2. Install MapLibre

MapLibre is a popular open-source library for rendering maps. Install it via npm:

3. Set Up a Basic Map

Here's an example of initializing a basic map in an HTML file:

4. Add Geolocation

To center the map on the user's current location, use the browser's Geolocation API:

5. Add a Search Function

Integrate a geocoding service for search functionality. For open-source solutions,

- [Nominatim](https://nominatim.org/) (OSM-based geocoding)
- [Pelias](https://pelias.io/)

6. Add Routing

Routing is key for navigation. Use an open-source routing engine like:

- [OSRM](http://project-osrm.org/)
- [Valhalla](https://github.com/valhalla/valhalla)

7. Add Turn-by-Turn Directions

For turn-by-turn navigation:

- Parse the OSRM route response, which contains steps and instructions.
- Display these instructions in a sidebar or overlay.

8. Test and Optimize

- Performance: Optimize map rendering and routing calls.
- Mobile UI: Make the app responsive for different devices.
- Offline Maps: Consider MapLibre's offline tile rendering features.

Tools and Resources

- Hosting Map Tiles: Use MapLibre or host your own tiles using TileServer-GL.
- Styling Maps: Use Maputnik to create custom styles.
- Documentation: Refer to the [MapLibre Docs](https://maplibre.org/).

Steps to Build a Mobile Navigation App with MapLibre

1. Choose Your Platform

- Native :
 - Android : Use MapLibre Native SDK with Java/Kotlin.
 - iOS : Use MapLibre Native SDK with Swift/Objective-C.
- Cross-Platform : Use React Native, Flutter, or similar frameworks for a single codebase.

2. Set Up Development Environment

- Install necessary tools:
 - For Android: Android Studio and Java/Kotlin.
 - For iOS: Xcode and Swift.
 - For Cross-Platform: Set up React Native or Flutter.
- Configure your project to use MapLibre libraries.

3. Integrate MapLibre

- Add MapLibre SDK or dependencies to your project.
- Set up permissions for location access and internet usage.
- Initialize the MapLibre map in your app with a default style.

4. Enable Geolocation

- Request location permissions from the user.
- Use platform-specific APIs to get the user's current location.
- Center the map on the user's position and add a location marker.

5. Add Map Features

- Enable map interaction (zoom, pan, and rotation).
- Add custom markers or overlays for points of interest.
- Allow the user to search for locations using a geocoding service.

6. Implement Routing

- Integrate a routing engine (OSRM, Valhalla, or GraphHopper).
- Fetch route data from the engine based on start and destination coordinates.
- Draw the route as a polyline on the map.

7. Add Navigation Features

- Turn-by-Turn Directions :
 - Parse routing instructions and display them.
 - Use text-to-speech for voice navigation.
- Traffic Layers :
 - Add real-time traffic overlays if available.

8. Optimize for Mobile

- Design a responsive UI optimized for touch.
- Cache map tiles for offline usage.
- Minimize API calls to reduce data usage and improve performance.

9. Test and Debug

- Test the app on multiple devices and simulators.
- Verify geolocation accuracy, route rendering, and navigation instructions.
- Debug any performance issues or crashes.

10. Deploy the App

- Prepare app for deployment by following platform-specific guidelines.
- Publish the app to Google Play Store (Android) and Apple App Store (iOS).

DİĞER PROJE BAŞLIKLARI

Herbir proje başlığı için bir araştırma yapılacak ve bir proje planı çıkarılacak. Proje staj süresi içinde belirlenen fonksiyonlar ile çalıştırılmaya çalışılacak.

Sahibinden benzeri bir site yapımı, classified sitesi

Finansal verilerin takibi için bir websitesi

Yapay zeka desteği ile web projeleri yapılması konusunda araştırma/inceleme yapılması ve örnek proje

Etkinlik takibi için harita tabanlı yazılım oluşturulması

Web tabanlı Uygulama Geliştirme

1. İhtiyaç Analizi ve Planlama

1. Hedef Belirleme

- Uygulama hangi sorunu çözecek veya hangi ihtiyacı karşılayacak?
- Hedef kitle (persona) kimdir? Bireysel kullanıcılar mı, kurumsal kullanıcılar mı?

2. Kullanıcı Hikayeleri (User Stories)

- Kullanıcıların uygulama içinde gerçekleştireceği senaryoları (örnek: "Kullanıcı giriş yapar ve profilini düzenler") kısa hikayeler halinde belirleyin.

3. Teknoloji Seçimi

- **Front-end:** React, Vue, Angular gibi modern çatıları; veya Svelte gibi daha yeni yaklaşımları değerlendirin.
- **Back-end:** Node.js, Python (Django/Flask/FastAPI), Java (Spring Boot), .NET gibi diller ve frameworkler arasından projenize uygun olanını seçin.
- **Veritabanı:** Veri modelinize göre ilişkisel (PostgreSQL, MySQL) ya da NoSQL (MongoDB, Redis) çözümleri düşünün.

4. Süreç ve Proje Yönetimi

- Agile (Scrum, Kanban) gibi çevik metodolojileri uygulayarak periyodik (sprint) geliştirmeler yapabilirsiniz.
 - Görev takibi için Trello, Jira, Asana gibi araçlar kullanın.
-

2. UI/UX ve Modern Tasarım Kriterleri

1. Responsive Design (Mobil Uyumluluk)

- Günümüzün büyük çoğunluğu mobil cihazlardan web'e eriştiği için, uygulamanızın farklı ekran boyutlarında sorunsuz çalışması esastır.
- Mobil öncelikli (Mobile-First) bir tasarım yaklaşımı benimseyin.

2. Erişilebilirlik (Accessibility – A11y)

- Renk kontrast oranlarını kontrol edin, metinlerin okunabilir olmasına dikkat edin.
- Ekran okuyucuların (screen reader) kolay algılayabilmesi için semantik HTML etiketlerini doğru kullanın (`<header>`, `<main>`, `<nav>`, `<section>`, `<button>` vs.).
- WCAG (Web Content Accessibility Guidelines) standartlarına mümkün olduğunca uyum sağlayın.

3. Performans Optimizasyonu

- Hızlı yükleme süreleri için resimleri optimize edin, aşırı büyük kütüphanelerden kaçının, mümkünse “lazy loading” teknikleri uygulayın.
- Lighthouse, PageSpeed Insights gibi araçlarla performans ve erişilebilirlik skorlarını ölçüp iyileştirmeler yapın.

4. Modern Arayüz Bileşenleri ve Tasarım Dilleri

- Design System mantığını benimseyin (örn. Material Design, Bootstrap, Tailwind, Chakra UI veya kendi özel tasarım kütüphaneniz).
- Tutarlı renk paleti, tipografi, ikonografi ve bileşen (component) stili kullanarak kurumsal kimliği veya uygulama ruhunu yansıtın.

5. Micro Interaction ve Animasyonlar

- Buton tıklamaları, geçişler ve formlar gibi bileşenlerde ufak animasyonlar ile kullanıcı deneyimini zenginleştirebilirsiniz.
- Aşırı veya gereksiz animasyonlardan kaçınarak performans dengesini koruyun.

3. Prototipleme ve Kullanıcı Geri Bildirimi

1. Wireframe / Mockup Hazırlama

- Figma, Sketch, Adobe XD, Framer gibi modern tasarım araçlarıyla sayfa taslakları (wireframe) ve düşük-orta-yüksek çözünürlüklü mockup'lar hazırlayın.
- Kullanıcı akışı (flow) net olsun: “Anasayfa → Giriş Formu → Profil Sayfası” gibi.

2. Clickable Prototip

- Gerçek kullanıcılardan veya ekip arkadaşlarınızdan, prototip üzerinde tıklanabilir tasarım öğeleriyle (giriş, geçiş vb.) erken geri bildirim alın.
- Önemli revizeleri bu aşamada yaparak geliştirme sürecindeki tekrarları azaltın.

3. Kullanıcı Testleri

- UI/UX sorunlarını erken tespit edebilmek için küçük gruplarla test yapın.
- Form validasyonları, navigasyon kolaylığı, sayfa yüklenme hızı gibi kritik noktaları ölçün.

4. Geliştirme Aşaması

1. Front-end Geliştirme

- **Yapılandırma:** Create React App, Vite, Next.js, Nuxt.js gibi modern tool'lar, projenin hızlı kurulumu için kolaylık sağlar.
- **Kod Organizasyonu:** Bileşen (component) tabanlı yapı kurun, karmaşayı önlemek için proje dosyalarını mantıklı klasörlere ayırın (örneğin `components`, `pages`, `services`, `utils` vb.).
- **State Yönetimi:** React için Redux, Context API, Zustand; Vue için Vuex/Pinia gibi araçlar veya yerleşik state management kullanın.
- **Test:** En azından unit test (Jest, Vitest vb.) ve bileşen testleri (React Testing Library, Cypress) ile kod kalitesini koruyun.

2. Back-end Geliştirme

- **API Tasarımı:** RESTful veya GraphQL API tercih edin. Versiyonlama (v1, v2) ve düzgün dokümantasyon (Swagger, Postman Collections vb.) ile bakım kolaylığı sağlayın.
- **Veritabanı ve ORM:** TypeORM, Sequelize, Prisma, Mongoose gibi veritabanı araçları ile tablolar/şemalar oluşturmayı otomatikleştirin.
- **Güvenlik:** API uç noktalarında kimlik doğrulama (JWT, OAuth2, vb.), giriş doğrulaması (input validation), rol bazlı erişim (RBAC) gibi yöntemler kullanın.
- **Test:** İş kurallarını test etmek için entegrasyon testleri (Mocha, Jest, PyTest vb.) yazarak kritik veritabanı işlemlerini güvenceye alın.

3. Mikroservis veya Tek Parça (Monolitik) Mimari

- **Monolitik:** Küçük projelerde hızlı başlangıç için uygundur.
- **Mikroservis:** Büyük projelerde ölçeklenebilirlik, farklı ekiplerin bağımsız çalışması ve farklı teknoloji yığınlarını kullanma imkânı sunar.

- **API Gateway:** Mikroservis yapısında servislerin iletişimini kolaylaştırmak için gateway katmanı kullanabilirsiniz.
-

5. Güvenlik ve Veri Koruma

1. HTTPS / SSL

- Geliştirme ortamında dahi HTTPS kullanmak, potansiyel güvenlik açıklarını erken fark etmenize yardımcı olur.
- Let's Encrypt gibi ücretsiz sertifika sağlayıcıları ile canlı ortamlarda SSL zorunlu hale getirin.

2. Veri Şifreleme

- Hassas verileri (örneğin şifreler, token'lar) mutlaka hash ve salt (BCrypt, Argon2 vb.) yöntemleriyle saklayın.
- Kullanıcı verileri üzerinde KVKK (GDPR) benzeri yasal düzenlemeler geçerliyse, bu kapsamda nelerin şifrelenip anonimleştirilmesi gerektiğini belirleyin.

3. Koruma Önlemleri

- XSS (Cross-Site Scripting), CSRF (Cross-Site Request Forgery), SQL Injection gibi yaygın saldırı türlerine karşı tedbir alın.
 - Güvenlik duvarları (WAF) ve düzenli güvenlik testleri (penetration test) yaparak zafiyetleri kapatın.
-

6. Dağıtım (Deployment) ve DevOps

1. CI/CD (Sürekli Entegrasyon / Sürekli Dağıtım)

- GitHub Actions, GitLab CI/CD veya Jenkins gibi araçlarla otomatik test ve dağıtım süreçlerini kurun.
- Kodlar birleştirildiğinde (merge) veya belirli bir branch'e push yapıldığında otomatik olarak test ve derleme (build) adımları çalışsın.

2. Sunucu / Hosting Seçimi

- Uygulama boyutuna ve bütçeye göre seçenekler:
 - AWS (EC2, Elastic Beanstalk, Lambda),
 - Azure (App Service),

- Google Cloud (App Engine),
- DigitalOcean Droplet veya Kubernetes.
- Static front-end yapılar (React, Vue vb.) için Netlify, Vercel gibi servisleri düşünebilirsiniz.

3. Konteyner ve Orkestrasyon

- Uygulama mikroservis yapısında veya hızlı ölçeklenecekse Docker ile konteynerize edin.
- Kubernetes (K8s) veya Docker Swarm ile otomatik ölçeklendirme ve yönetim kolaylığı elde edebilirsiniz.

4. Versiyon Kontrol ve Ortam Yönetimi

- Geliştirme (development), test (staging) ve üretim (production) ortamlarını ayrı tutun.
- Ortam dosyalarındaki API anahtarları, veritabanı bilgileri vb. için `.env` veya bir şifreleme/secret yönetim sistemi kullanın.

7. Sürekli İyileştirme ve Ölçümleme

1. Performans İzleme

- Uygulamanın canlı ortamda performansını ölçmek için APM (Application Performance Monitoring) araçları (New Relic, Datadog, Elastic APM) veya Google Analytics/GA4, Segment gibi araçları kullanın.

2. Loglama ve Hata Takibi

- Sunucu loglarını merkezi bir sistemde (Logstash, Kibana, Splunk) toplayın.
- Hatalar için Sentry, Rollbar gibi gerçek zamanlı hata izleme hizmetlerini entegre edin.

3. Kullanıcı Geri Bildirimleri

- Uygulama içerisinde kullanıcıların hata raporu, geliştirme önerisi vb. sunabileceği kanallar oluşturun.
- Toplanan geri bildirimlere göre sprint bazlı iyileştirmeler yapın.

4. Düzenli Güncellemeler

- Bağımlılık (dependency) yönetimi yaparak kütüphaneleri güncel tutun. Güvenlik yamalarını kaçırmayın.

- Yeni özelliklerin eklenmesi veya mevcut özelliklerin iyileştirilmesi için yol haritası (product roadmap) oluşturun.

Özet

1. **Planlama ve Analiz:** Projenin amacını, hedef kitlesini ve teknik altyapı seçimini doğru yapın.
2. **Modern Tasarım:** Responsive, erişilebilir ve performans odaklı bir arayüz oluşturun. Design system ve UI bileşenlerini tutarlı kullanın.
3. **Prototip ve Kullanıcı Testi:** Mockup'lar ve prototipler ile hızlı geri bildirim alarak tasarımı olgunlaştırın.
4. **Geliştirme:** Front-end ve back-end yapısını modüler ve güvenli bir şekilde inşa edin. API tasarımını ve veri modelini doğru kurgulayın.
5. **Güvenlik:** SSL, veri şifreleme, XSS/CSRF koruması gibi önlemleri ihmal etmeyin.
6. **Dağıtım ve DevOps:** CI/CD süreçleri kurarak uygulamayı sürekli entegre ve dağıtılabilir hale getirin. Bulut servisleri ve konteyner teknolojilerinden yararlanın.
7. **Sürekli Ölçümleme ve İyileştirme:** Performans ve hata analizini düzenli yaparak kullanıcı geri bildirimleriyle ürünü geliştirmeye devam edin.

Bu adımları takip ederek, hem **çağdaş tasarım ilkelerine** uygun hem de **güvenli ve ölçeklenebilir** bir web uygulaması geliştirebilirsiniz. Projenizi planlı ve iteratif bir yaklaşımla ilerletmek, olası sorunları erken aşamalarda fark etmenizi ve kaliteli bir sonuç elde etmenizi kolaylaştıracaktır. Kolay gelsin!

JavaScript

1. Temel Bilgileri Güçlendirin

1. HTML ve CSS Bilgisi

JavaScript, web geliştirme için büyük oranda HTML ve CSS ile iç içe çalışır. Temel HTML ve CSS bilginiz olmadan JavaScript ile etkileşimli web sayfaları oluşturmak zorlaşır. Bu nedenle, öncelikle HTML ve CSS temellerini öğrenmiş olduğunuzdan emin olun.

2. Temel JavaScript Yapısı

- Değişken tipleri (var, let, const)
 - Veri türleri (number, string, boolean, object, array)
 - Operatörler (+, -, *, /, %, =, ==, ===, vb.)
 - Koşullar (if, else, switch)
 - Döngüler (for, while, do...while)
 - Fonksiyonlar (function declaration, arrow function, anonymous function)
3. Bu konuları mümkün olduğunca kısa zamanda genel hatlarıyla kavramaya odaklanmak, sonrasında uygulamalarda geliştire geliştire derinleşmek sizi hızlandırır.

2. Öğrenme Stratejileri

1. Bir Kaynaktan “Hızlı Başlangıç” Yapın

Öncelikle, çok detaylı ve uzun açıklamalardan oluşan kaynaklara boğulmadan, temel kavramları hızlıca gözden geçirebileceğiniz bir “JavaScript hızlı başlangıç” rehberiyle başlayın. Bu rehberler genellikle kısa tanımlarla, örneklerle hızlıca ilk projeleri yaptırmayı amaçlar.

2. Bol Bol Uygulama ve Küçük Projeler

Teoriyi çok uzun süre okumak yerine, her yeni öğrendiğiniz konuyu hemen küçük uygulamalara dönüştürün.

- Butona tıklandığında bir metnin ekranda değişmesini sağlayın.

- Basit bir hesap makinesi uygulaması yapın.
- Array üzerinde basit işlemler (toplama, filtreleme vb.) uygulayın.
- 3. Bu tip minik projelerle JavaScript'in mantığını kavramak hızlı ilerlemenizi sağlar.
- 4. **Online Alıştırma / Kod Kırma (Debugging) Siteleri**
CodePen, JSFiddle gibi siteler, hemen tarayıcı üzerinde JavaScript kodlarınızı test edip anlık sonuç almanıza imkân tanır. Özellikle, bir fikri veya örneği hızlıca deneyip görmek için bu araçlar zaman kazandırır.
- 5. **Temel Algoritma Alıştırmaları**
Temel algoritma soruları ve veri yapısı örneklerini çözmek, dilin temellerini çabuk pekiştirmenize yardımcı olur. Örneğin, "reverse a string" (bir metni ters çevirme), "fizzbuzz" (belirli sayı aralıklarındaki koşullu çıktı), "faktöriyel hesaplama" gibi temel örnekler, sözdizimi ve mantık pratiği yapmanıza olanak tanır.
- 6. **Proje Tabanlı Öğrenme**
Biraz temel oturduktan sonra "To-do list", "quiz uygulaması" gibi basit ama uçtan uca bir proje geliştirmeye çalışmak, öğrendiklerinizi kalıcı hale getirmenin en iyi yoludur.

3. Kaynak Önerileri

Ücretsiz Online Kaynaklar

1. **MDN Web Docs (Mozilla)**
 - <https://developer.mozilla.org/tr/docs/Web/JavaScript>
JavaScript için resmi ve detaylı bir dokümantasyon sağlar. Türkçe içerikler de bulunmaktadır. Temelden ileri seviyeye kadar birçok konuya derinlemesine bakabilirsiniz.
2. **W3Schools**
 - <https://www.w3schools.com/js/>
Hızlı başlangıç rehberleri ve uygulamalı örnekler mevcuttur. Etkileşimli "Try it yourself" bölümleriyle kolayca deneyebilirsiniz.
3. **FreeCodeCamp**

- <https://www.freecodecamp.org/>
JavaScript temellerinden algoritma alıştırmalarına kadar adım adım yönlendiren etkileşimli bir platformdur.

Video Dersleri (YouTube)

1. Traversy Media

Basit, anlaşılır anlatımlarıyla HTML, CSS ve JavaScript için başlangıç seviyesinden proje tabanlı eğitimlere kadar çeşitli videolar sunar.

2. Fireship

Çok kısa ve hızlı anlatımlar yaparak hızlı bir şekilde konuya hâkim olmanızı sağlar. Özellikle “JavaScript in 100 seconds” gibi hızlı özet videolarla temel kavramları pekiştirebilirsiniz.

3. Türkçe Kanallar

Engin Demiroğ, Adil Altaş gibi eğitmenlerin YouTube kanallarında ücretsiz Türkçe JavaScript dersleri bulabilirsiniz.

Online Kurs Platformları

1. Udemy

- “Modern JavaScript”, “JavaScript: The Complete Guide” vb. kapsamlı ve proje tabanlı kurslar bulabilirsiniz. Türkçe ve İngilizce ders seçenekleri mevcuttur.
- Özellikle Türkçe olarak anlatılmış, yüksek puanlı ve fazla sayıda öğrenciye sahip kurslar, başlangıç için hızlı ilerlemenizi sağlayabilir.

2. Codecademy

JavaScript için etkileşimli dersler sunar, konsol üzerinde anlık kod çalıştırma imkânı verir.

Kitaplar

1. Eloquent JavaScript (Marijn Haverbeke)

- Online olarak da ücretsiz erişilebilen (İngilizce) bir kitap. Temelleri ve algoritmik düşüncüyü iyi anlatır.
- Türkçe çevirileri de bazı topluluk forumlarında mevcuttur.

2. You Don’t Know JS (Kyle Simpson)

- Serinin bazı bölümleri zorlayıcı olabilir ancak dilin çalışma mantığını derinlemesine öğrenmek isteyenler için çok faydalıdır.
 - Ücretsiz olarak GitHub'da erişilebilir (İngilizce).
-

4. Hızlı Öğrenmek için Ek İpuçları

1. Sık Yapılan Hatalarla Yüzleşme

JavaScript'te en çok karıştırılan konular (örneğin `==` vs `===`, `var` vs `let` vs `const`, scope, hoisting, async/await) üzerine odaklanın. Bu konuları iyi anlarsanız pek çok hatayı en başından önlemiş olursunuz.

2. Tarayıcı Konsolunu Aktif Kullanın

Herhangi bir site veya projeniz açıkken **F12** ile Geliştirici Araçları'nı (Developer Tools) açarak konsolda hızlı testler yapabilirsiniz. Özellikle "Console" sekmesi, JavaScript öğrenirken çok yardımcı olur.

3. Kod Okuma ve Debugging Alıştırmaları

Hazır projelerin kodlarını incelemek, hataları düzeltmeye çalışmak, öğrenme sürecinizi hızlandırır. GitHub'da basit JavaScript projelerini inceleyerek, neyin nasıl yapıldığını görmeye çalışın.

4. Topluluklara Katılın

Stack Overflow, GitHub, Reddit gibi ortamlarda soru sorarak veya yanıt arayarak pratik yapabilirsiniz. Türkçe forumlar da (örneğin, r/TurkeyProgrammers gibi) yardımlaşmak açısından faydalıdır.

5. Düzenli ve Sürekli Pratik

Her gün en az 15-30 dakika pratik yapmak, beynin konuları sindirmesi için çok etkilidir. İstikrarlı ve düzenli öğrenme, hızlı ilerlemenin anahtarıdır.

Sonuç

- **Hızlı öğrenme** için en önemli unsur, kısa teoriden sonra **hemen uygulamaya** geçmek ve küçük projelerle pratik yapmaktır.
- MDN, W3Schools, FreeCodeCamp gibi ücretsiz ve kaliteli dokümanları kullanarak hızla temel bilgilere hâkim olabilir, hemen ufak projeler deneyerek öğreneceğiniz bilgileri pekiştirebilirsiniz.

- Udemy gibi platformlardan Türkçe veya İngilizce hızlı başlangıç kursları seçerek daha sistematik bir öğrenme rotası izlemek, dağınık konularla boğuşmanızı engeller ve motivasyonunuzu yüksek tutar.

Bu strateji ve kaynaklarla planlı ve düzenli bir şekilde çalışarak JavaScript'i kısa sürede etkili biçimde öğrenebilirsiniz. Başlangıçta karmaşık gözüken noktalar, düzenli pratikle hızla netleşecektir. Kolay gelsin!

React

1. React'e Başlamadan Önce Bilmeniz Gerekenler

1. HTML ve CSS Temelleri

React bileşenlerini geliştirmek için temel düzeyde HTML ve CSS bilginizin olması gerekir. Tasarımsal kısımlarda ve öğelerin sayfaya yerleştirilmesinde bu bilgi işinize yarayacaktır.

2. Modern JavaScript / ES6+

React, ES6 (ve üstü) JavaScript özelliklerini aktif olarak kullanır. Dolayısıyla, özellikle aşağıdaki konuları yeterince anlamış olmak hız kazandırır:

- `const`, `let` değişken tanımları
- Arrow functions `() => {}`
- Array ve object destructuring
- Spread operator `...`
- Map, filter gibi dizi metodları
- Promise, `async/await` yapıları (veri çekme işlerinde sık sık kullanacaksınız)

3. Node.js ve npm (yarn) Temel Bilgisi

React projeleri genellikle Node.js altyapısını kullanır ve paketleri indirmek için npm veya yarn aracına ihtiyaç duyarsınız. Bu sebeple, Terminal/CLI üzerinde en azından şu komutları çalıştırabilir olmanız çok faydalıdır:

- `npm init`, `npm install`, `npm run start`, `npm run build`

2. React'in Temel Kavramları

1. Bileşenler (Components)

React'in temel yapıtaşı bileşenlerdir. Her bileşen, kendi içinde görsel arayüzün bir parçasını temsil eder. Fonksiyonel bileşenleri (function components) kullanmak artık daha yaygın.

2. JSX (JavaScript XML)

React'te bileşenler içerisinde HTML benzeri bir sözdizimiyle (JSX) kod yazılır. Bu sözdizimin tam olarak JavaScript içinde dönüştürüldüğünü (transpile)

anlamanız çok önemlidir.

3. Props (Properties)

Bileşenlere dışarıdan veri aktarımı için kullanılır. Bir bileşen kendisine gönderilen props'ları alarak dinamik içerik oluşturur.

4. State (Durum Yönetimi)

Bir bileşenin kendi içindeki durumu, bileşenin nasıl görüldüğünü ve nasıl davrandığını belirler. React'te `useState` ile fonksiyonel bileşenlerde durum yönetimi yapılır.

5. Hook'lar

React 16.8 sonrası, fonksiyonel bileşenlerde state ve yaşam döngüsü özelliklerini kullanabilmemizi sağlayan fonksiyonlardır. Öne çıkan bazı hook'lar:

- `useState`
- `useEffect`
- `useContext`
- `useReducer`

6. Lifecycle (Yaşam Döngüsü) Mantığı

Fonksiyonel bileşenlerde `useEffect` hook'u ile "componentDidMount, componentDidUpdate, componentWillUnmount" gibi yaşam döngüsü aşamalarını kontrol edebilirsiniz.

7. Router ve Veri Çekme

- React Router kullanarak sayfalar arası gezinmeyi (routing) yapılandırabilirsiniz.
- Veri çekmek için `fetch`, `axios`, vs. kullanıp yanıtları state'e atayabilirsiniz.

3. Öğrenme Stratejileri

1. Hızlı Başlangıç Kılavuzunu İzleyin

- React'in [resmî dokümantasyonundaki](#) "Quick Start" bölümünü veya Türkçe çevirilerini okuyarak bir proje oluşturma, basit bir bileşen yazma ve props kullanma gibi temel adımları hızlıca deneyin.

- Ayrıca Create React App veya Vite gibi hazır yapılandırma araçlarıyla sıfırdan ayar yapmakla vakit kaybetmeden anında proje başlatabilirsiniz.

2. Küçük Örneklerle Pratik

Her yeni kavramı (props, state, hook, vb.) ufak projeler içinde deneyin:

- Liste oluşturma ve filtreleme
- Form verisi alma
- Sayaç uygulaması (Counter)
- Basit “to-do list” uygulaması

3. Proje Tabanlı Yaklaşım

Temel kavramları öğrenir öğrenmez, uçtan uca basit bir proje yapmaya başlayın. Örneğin:

- Haber uygulaması (API’den veri çekerek listeleme)
- Film/kitap arama uygulaması (kullanıcıdan arama girdisi alıp API çağırısı yaparak sonuçları gösterme)
- Blog veya not defteri uygulaması (React Router’ı ve state yönetimini kullanarak ekleme, düzenleme)

4. Hataları ve Uyarıları Okuma

React genelde hata ve uyarıları net bir şekilde konsolda gösterir. Öğrenme sürecinde karşılaştığınız hataları araştırarak (örneğin “Error: Hooks can only be called inside of the body of a function component” gibi) hızla çözmek, konuları kalıcı hale getirir.

5. State Yönetimi Kütüphaneleri

Temel React’i kavradıktan sonra Redux, Zustand, Context API gibi farklı state yönetimi yaklaşımlarını öğrenebilirsiniz. Ancak çok hızlı başlangıç aşamasında bu kütüphanelere boğulmadan önce React’in kendi mantığını iyi öğrenmek daha önemli.

4. Kaynak Önerileri

Resmî Dokümantasyon

1. React Docs (Yeni Sürüm – react.dev)

[https://react.dev/](https://react.dev)

React’in yeni dokümantasyonu oldukça kullanıcı dostu. Hem temel kavramları hem de ileri seviye konuları iyi örneklerle açıklıyor.

Ücretsiz Online Kaynaklar

1. Scrimba

<https://scrimba.com/> üzerinde React temellerini kapsayan etkileşimli dersler bulabilirsiniz. Kod ve video aynı ekranda, anında deneyim sunar.

2. FreeCodeCamp

- React bölümü de olan kapsamlı bir platform.
- Ufak alıştırmalardan proje tabanlı büyük çalışmalara kadar geniş bir içerik sunuyor.

3. W3Schools

- <https://www.w3schools.com/react/>
Kısa ve hızlı örneklerle başlangıç düzeyine yardımcı olur.

Video Dersleri (YouTube)

1. Traversy Media

İngilizce React dersleri çok net ve proje tabanlı anlatımları bulunuyor. Hızlı ilerlemek isteyenler için oldukça uygun.

2. Fireship

Kısa ve hızlı özetler sunuyor (örneğin “React in 100 seconds”). React’in temel fikrini hızlı kavramak için idealdir.

3. Türkçe Kaynaklar

- Engin Demiroğ, Adil Altaş, Kodluyoruz gibi kanallarda ücretsiz React dersleri veya canlı yayınlar bulabilirsiniz.
- Ayrıca Patika.dev platformu üzerinden çeşitli React eğitimleri, projeleri mevcuttur.

Online Kurs Platformları

1. Udemy

- Modern React kursları ile sıfırdan proje tabanlı öğrenme yapabilirsiniz.
- Türkçe veya İngilizce kurslardan yüksek puanlı, güncel içerikleri tercih edin. (React sürümlerinin güncelliğine dikkat edin.)

2. Codecademy

- Etkileşimli olarak adım adım React dersleri sunar.

- Ücretsiz bölümleriyle hızlı başlangıç yapabilirsiniz.
-

5. Hızlı Öğrenmek için Ek İpuçları

1. Sık Kullanılan Hataları Araştırın

- React kullanırken en çok yapılan hataları, “React common pitfalls” gibi listelerden inceleyin. Böylece önceden bilgi sahibi olabilir ve zamanı verimli kullanabilirsiniz.

2. Dev Tools’u Kullanın

- Tarayıcıda React Developer Tools eklentisiyle bileşenlerin durumunu inceleyip state/props kontrolünü yapmak öğrenmeyi çok hızlandırır.

3. Uygulama ve Eğlenceli Projeler

- Basit projeleri tamamladıktan sonra ufak bir oyun veya bir quiz uygulaması yapmak eğlenceli ve öğretici olur.

4. Düzenli Kodlama ve Paylaşım

- Kodlama pratiğine her gün vakit ayırın (15-30 dakika bile olsa).
- Öğrendiğiniz projeleri GitHub’da paylaşarak geri bildirim alabilirsiniz.

5. Topluluklarda Aktif Olun

- Stack Overflow, Reddit gibi topluluklarda soru sorarak veya başkalarının sorularına göz atarak React’in nasıl kullanıldığına dair canlı örnekler görebilirsiniz.
-

Sonuç

- React’i **kısa sürede öğrenmek**, öncelikle iyi bir **JavaScript** temeline ve **bol pratik** yapmaya bağlıdır.
- Resmî dokümantasyon (react.dev) ve kısa proje tabanlı anlatım sunan kurslar (Udemy, Scrimba, FreeCodeCamp vb.) hızlı başlangıç için idealdir.
- Öğrendiklerinizi mutlaka **küçük projelere** uygulayın. Böylece hem React’in mantığını kavramak hem de hataları daha çabuk düzeltmek kolaylaşacaktır.
- Çok karmaşık kütüphanelere dalmadan önce React’in kendi “state & props” ve “component yapısı” mantığını iyice oturtmaya odaklanın. Bu önerilere uyarak planlı bir şekilde pratik yaparsanız, React’i kısa sürede kullanabilir hâle gelebilirsiniz. Kolay gelsin!

Yazılım Tasarım Desenleri

<https://www.btkakademi.gov.tr/portal/course/player/deliver/yazilim-tasarim-desenleri-12150>